CLOUD & PLATFORMS

Enterprise Integration Patterns

Messaging patterns, SAP and SaaS connectivity, reliability and cutover — the plumbing that is 70% of every enterprise deployment.

integration · enterprise · messaging · kafka · rabbitmq · webhooks · sap · idoc · odata · soap · salesforce · cdc · debezium · idempotency · ipaas · cutover · fde

Study Pack · Comprehensive reference & interview prep · ~130 min read · 28,693 words

CONTENTS

1. The Integration Reality of Enterprise Customers
2. The Four Integration Styles
3. Synchronous versus Asynchronous
4. The Messaging Patterns Toolkit
5. Protocols and Formats You Will Actually Meet
6. Webhooks
7. Message Brokers and Event Streaming
8. Reliability Engineering for Integrations
9. Connecting to SAP
10. Connecting to the Big SaaS Platforms
11. Legacy and the Long Tail
12. Databases as Integration Points
13. iPaaS and Middleware
14. Security and Governance of Integrations
15. Rate Limits, Quotas and Designing Within a Budget
16. Testing, Cutover and Operations
17. Designing an Integration a Customer Will Approve
18. Interview Questions
19. Integration Design Checklist and Lab Plan

The Integration Reality of Enterprise Customers

The demo took four weeks. The deployment takes nine months. Nothing about the model, the algorithm or the user interface accounts for that gap — it is almost entirely plumbing. Getting data out of an ERP that was configured in 2011 by a consultancy that no longer exists, through a proxy that strips headers, into a schema that three departments disagree about, on a schedule that fits a change window that happens twice a year. That work is the job. A Forward Deployed Engineer who is good at integration is worth several who are good at everything else.

~70%
OF DEPLOYMENT EFFORT IS INTEGRATION

5-12
SYSTEMS IN A TYPICAL ENTERPRISE FLOW

>50%
OF "BUGS" ARE DATA-SHAPE MISMATCHES

1
CHANGE WINDOW PER QUARTER, IF YOU ARE LUCKY

What an enterprise landscape actually looks like

The architecture diagram the customer shows you in the kickoff has eight boxes. The real system has sixty, and the important ones are not on the diagram because the people who drew it did not know about the Access database in Finance or the nightly SFTP drop that keeps the warehouse running. Expect to find:

SYSTEMS OF RECORD

SAP ECC or S/4HANA, Oracle EBS, Microsoft Dynamics, Workday, a custom mainframe application. Slow to change, owned by a team that does not report to your sponsor, protected by a change board.

SYSTEMS OF ENGAGEMENT

Salesforce, ServiceNow, Microsoft 365, a customer portal, a mobile app. Faster-moving, API-friendly, and usually where your sponsor sits.

MIDDLEWARE

MuleSoft, Boomi, SAP CPI, Azure Logic Apps, an IBM Integration Bus from 2009, or an in-house Java ESB that one person understands. You will be told to go through it.

THE DATA PLATFORM

A data warehouse (Snowflake, BigQuery, Synapse, Teradata), a lake, and an overnight ETL that everyone depends on and nobody owns end to end.

THE SHADOW ESTATE

Excel files on SharePoint that are load-bearing, an Access database, a Python script on someone's desktop, a Power Automate flow built by a business analyst. These are often the actual integration points.

THE SECURITY LAYER

An API gateway, a WAF, a forward proxy, a TLS-inspecting middlebox, an identity provider, a PAM vault, a DLP agent. Each is owned by a different team with a different queue.

The questions to ask in the first week

1. **What is the system of record for this entity?**
   Customer, product, employee, invoice, case — there must be exactly one authoritative source per entity, and if there are two, you have found the project's hardest problem on day one. Ask who arbitrates when they disagree.

2. **How do I get data out, and who owns that interface?**
   Not "does it have an API" — specifically: which API, which version, which environment, who grants access, what is the lead time for a service account, and when was it last used successfully by anyone.

3. **What is the volume and the shape?**

Rows per day, peak hour, average and maximum record size, how far back the history goes, how many of them change per day. A design for 10,000 records a day is a different design from one for 10 million.

4 **How fresh does the data need to be?**

Real time, minutes, hourly, nightly? The honest answer is usually much looser than the first answer, and the difference between "real time" and "within fifteen minutes" is often the difference between a streaming architecture and a cron job.

5 **What happens when it fails?**

Who notices, who fixes it, is a partial result acceptable, can we replay, is double-processing dangerous? Answer this before you design anything.

6 **What is the change process?**

How long from "we need a firewall rule" to "the firewall rule exists"? How many change windows per quarter? Who signs off? This determines your project plan more than any technical factor.

7 **Who has done this before here?**

There is almost always an existing integration with the same system. Find it, read it, and copy its auth pattern. It has already survived the security review.

✦ **Find the person, not the documentation**

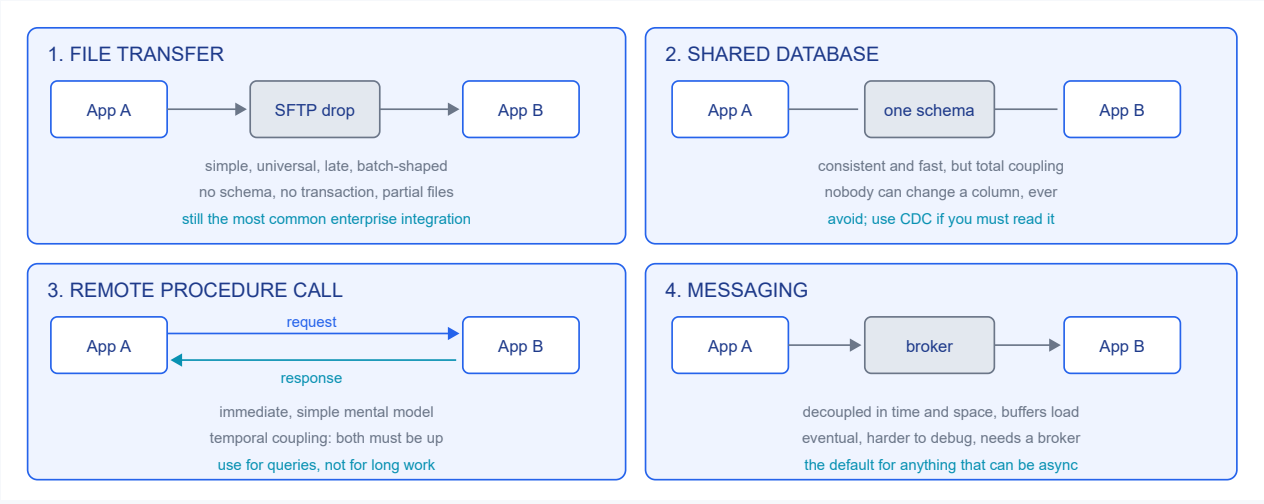
Every enterprise has one person per system who actually knows how it works — usually not the architect, usually someone with "senior analyst" in their title who has been there eleven years. Find them in week one, buy them a coffee, and ask them what breaks. Thirty minutes with that person is worth three weeks of reading Confluence pages last updated in 2019. In SAP landscapes this person is in the Basis team; in Salesforce it is the admin; in the mainframe world they are usually about to retire, which is its own kind of deadline.

⚠ **The integration tax nobody budgets for**

Every integration point adds: a credential that expires, a schema that drifts, a rate limit you will hit, a maintenance window you do not control, a network path that can break, and a team you must coordinate with. Five integration points is not five times the work of one — it is more, because the failure modes combine. When scoping, count integration points, not features, and push hard to reduce the count. The best integration is the one you talked the customer out of needing.

The Four Integration Styles

Hohpe and Woolf identified four fundamental ways to connect applications, and twenty-plus years later the taxonomy is unchanged because it is grounded in physics rather than fashion. Every integration you build is one of these four, or a combination. Knowing the trade-offs lets you argue for the right one instead of defaulting to whatever you used last time.



The four integration styles. Most real enterprise flows combine two or three.

File transfer

One application writes a file; another reads it. It is the oldest style, it is universally supported, and it remains overwhelmingly the most common way enterprises move data between organisations. Every bank, insurer, retailer and manufacturer you will work with has dozens of nightly SFTP jobs running right now. Do not be snobbish about it: file transfer has genuine virtues. It requires no shared runtime, no compatible library versions, no synchronised uptime, and it is trivially auditable — the file is right there, you can open it, you can reprocess it, and the security team understands it.

Strength	Weakness	Mitigation
Works through any firewall that allows one port	Latency is the batch interval, minimum	Increase frequency; drop files every 15 min rather than nightly
No shared technology stack	No schema enforcement; a column shifts and everything breaks silently	Header validation, column-count assertion, a schema file alongside
Natural audit trail and replay	Partial files: the reader picks up a file mid-write	Write to `name.tmp`, atomic rename to `name.csv`; or a `.done` sentinel file
Easy to reconcile — count the rows	No transaction semantics across files	Control file with record count and checksum; reject the batch if they disagree
Scales to huge volumes cheaply	Duplicate delivery if a job reruns	Idempotent load keyed on a business key; archive processed files by name

PYTHON

```python
# A file-drop consumer that survives contact with reality.
# Every one of these guards exists because of a real production incident.
import csv, hashlib, os, shutil, time
from pathlib import Path

INBOX, WORK, ARCHIVE, ERROR = map(Path, ("/data/inbox", "/data/work",
                                         "/data/archive", "/data/error"))
EXPECTED_HEADER = ["invoice_id", "vendor_id", "amount", "currency",
                   "invoice_date", "due_date", "cost_centre"]

def stable(path: Path, quiet_seconds: float = 5.0) -> bool:
    """Guard 1: do not read a file that is still being written."""
    s1 = path.stat()
    time.sleep(quiet_seconds)
    s2 = path.stat()
    return s1.st_size == s2.st_size and s1.st_mtime == s2.st_mtime

def process_drop(path: Path):
    # Guard 2: sentinel file. The producer writes DATA.csv then DATA.csv.done
    sentinel = path.with_suffix(path.suffix + ".done")
    if not sentinel.exists():
        return                                  # not finished yet

    if not stable(path):
        log.warning("file.still_growing", file=path.name)
        return

    # Guard 3: claim the file by MOVING it. Two workers cannot both take it.
    work = WORK / path.name
    try:
        path.rename(work)                       # atomic within a filesystem
        sentinel.unlink(missing_ok=True)
    except FileNotFoundError:
        return                                  # another worker won the race

    digest = hashlib.sha256(work.read_bytes()).hexdigest()
    batch_id = f"{work.stem}:{digest[:16]}"

    # Guard 4: idempotency. A rerun of yesterday's job must not double-post.
    if db.batch_already_processed(batch_id):
        log.warning("file.duplicate_batch", batch_id=batch_id, file=work.name)
        shutil.move(work, ARCHIVE / f"{work.name}.dup")
        return

    # Guard 5: control file. Count and checksum must match before ANY row lands.
    ctrl = INBOX / f"{work.stem}.ctrl"
    expected_rows = expected_sum = None
    if ctrl.exists():
```

```
        meta = dict(line.strip().split("=", 1) for line in
                    ctrl.read_text().splitlines() if "=" in line)
        expected_rows = int(meta.get("RECORD_COUNT", -1))
        expected_sum  = meta.get("SHA256")
        if expected_sum and expected_sum.lower() != digest:
            fail(work, batch_id, "checksum_mismatch",
                 expected=expected_sum, actual=digest)
            return

    rows, errors, total_amount = [], [], 0
    with work.open(newline="", encoding="utf-8-sig") as fh:   # BOM: Excel's gift
        reader = csv.DictReader(fh)
        # Guard 6: schema. A silently added column is the classic 3am incident.
        if reader.fieldnames != EXPECTED_HEADER:
            fail(work, batch_id, "header_mismatch",
                 expected=EXPECTED_HEADER, actual=reader.fieldnames)
            return
        for lineno, row in enumerate(reader, start=2):
            try:
                rows.append(validate_invoice(row))
                total_amount += int(round(float(row["amount"]) * 100))
            except Exception as e:
                errors.append({"line": lineno, "error": str(e),
                               "invoice_id": row.get("invoice_id")})

    if expected_rows is not None and len(rows) + len(errors) != expected_rows:
        fail(work, batch_id, "row_count_mismatch",
             expected=expected_rows, actual=len(rows) + len(errors))
        return

    # Guard 7: all-or-nothing threshold. 3 bad rows in 10,000 is normal;
    # 4,000 bad rows means the upstream changed and we must NOT load any.
    if errors and len(errors) / max(len(rows) + len(errors), 1) > 0.05:
        fail(work, batch_id, "too_many_invalid_rows",
             invalid=len(errors), total=len(rows) + len(errors),
             sample=errors[:5])
        return

    # Guard 8: one transaction per file, with the batch recorded inside it.
    with db.transaction():
        db.bulk_upsert_invoices(rows)
        db.record_batch(batch_id, file=work.name, rows=len(rows),
                        rejected=len(errors), amount_minor=total_amount,
                        sha256=digest)

    if errors:
        write_reject_file(ERROR / f"{work.stem}.rejects.csv", errors)
    shutil.move(work, ARCHIVE / f"{time.strftime('%Y/%m/%d')}/{work.name}")
```

```
    log.info("file.processed", batch_id=batch_id, rows=len(rows),
             rejected=len(errors), amount_minor=total_amount)
```

> ✕ **The eight ways a file drop fails in production**
> Partial read of a file still being written. A rerun that double-posts every invoice. A column silently added or reordered by the upstream team. UTF-8 BOM, CRLF line endings, or a Windows-1252 encoding that makes every accented name a question mark. A field containing an unescaped delimiter — the classic being a company name with a comma. Numbers in scientific notation because the file passed through Excel. An empty file arriving because the upstream job failed but still wrote its output. And the file that simply never arrives, which produces no error at all unless you alert on its absence. Build a guard for each one, and alert on *expected file did not arrive by 06:00* — missing data is the failure nobody detects.

Shared database

Two applications read and write the same tables. It is seductive — no serialisation, no network hop, transactional consistency for free — and it is almost always a trap. The schema becomes a shared public API that nobody can version, so no team can change a column without coordinating with every other team, and within two years nobody dares change anything. You will meet this pattern constantly in older estates; your job is usually to avoid extending it.

> ⚠ **When a shared database is acceptable**
> Read-only access to a replica, for reporting or for a read model you rebuild from scratch, with a documented understanding that the schema may change and you will break. Or a genuinely shared reference dataset with a stable, versioned schema and a single writer. Everything else — especially two applications writing to the same table — accumulates integrity problems that surface as data corruption two years after you have left.

Remote procedure call

One application calls a function in another and waits for the answer. REST, SOAP, gRPC, GraphQL, SAP RFC — all RPC in different clothes. The mental model is simple, which is its strength and its danger: it makes a network call look like a local call, hiding the fact that it can be slow, can fail partially, can time out after doing the work, and requires both systems to be available simultaneously. That last property, **temporal coupling**, is the one that hurts in enterprise environments where the other system has a maintenance window you do not control.

Messaging

One application puts a message on a channel; another takes it off, possibly much later. This decouples the two in time (the consumer can be down), in space (the producer need not know who consumes), and in throughput (the queue absorbs bursts). The costs are real: you need a broker, the semantics are eventual rather than immediate, duplicate delivery is a fact of life you must design for, and debugging a flow you cannot follow synchronously is harder. In exchange you get a system where one component's outage degrades rather than destroys.

Criterion	File	Shared DB	RPC	Messaging
Coupling	Low (format only)	Total	Medium (interface + time)	Low (message contract)
Latency	Batch interval	Immediate	Immediate	Seconds
Both systems must be up	No	Yes	Yes	No
Handles load spikes	Yes	No	No	Yes
Transactional	No	Yes	No (needs saga)	No (needs outbox/idempotency)
Firewall friendliness	Excellent	Poor	Good (443)	Medium (broker ports)
Debuggability	Excellent	Good	Good	Harder — needs tracing
Typical enterprise verdict	"We already do this"	"Security will say no"	"Fine, through the gateway"	"Which broker? We have three"

Synchronous versus Asynchronous

The single most consequential design decision in any integration is whether the caller waits. Get it wrong in the synchronous direction and you build a system that collapses when a dependency slows down. Get it wrong in the asynchronous direction and you

build complexity nobody needed for a call that takes eighty milliseconds. There is a decision procedure, and it takes about two minutes.

The decision procedure

1. **Does a human need the answer right now to continue?**
   If yes, synchronous — but only for the part they need. Validate synchronously, process asynchronously.

2. **How long does the work take, honestly, at p99?**
   Under about one second, synchronous is fine. Over about five seconds, a synchronous HTTP call is a liability: gateways time out, browsers give up, connections are held. Over thirty seconds, synchronous is indefensible.

3. **Can the work be retried safely?**
   If retries are safe and cheap, asynchronous becomes much more attractive, because the queue gives you retry semantics for free.

4. **What is the volume profile?**
   Smooth and modest, synchronous is fine. Bursty — 50,000 records at 02:00 on the first of the month — and you need a queue to flatten it, because your downstream is sized for the average and the burst will break it.

5. **What is the availability of the downstream?**
   If it has a four-hour maintenance window every Sunday, a synchronous design means four hours of failures. A queue means four hours of accumulation and a catch-up.

6. **Does the customer's network allow it?**
   You may not be allowed to receive inbound HTTP at all, which removes several options immediately.

✦ **The pattern that resolves most arguments**

Accept synchronously, process asynchronously, report status by polling or callback. `POST /jobs` validates the request in 50ms, writes it to a queue, returns `202 Accepted` with a `Location: /jobs/{id}` header. The client polls that URL or receives a webhook. The user gets immediate confirmation that their request was received and valid, the heavy work happens in a system you can scale and retry independently, and no gateway timeout can ever affect it. This design handles both "I need to know it worked" and "this takes four minutes" simultaneously, and it is what I propose by default.

PYTHON

```python
from fastapi import FastAPI, HTTPException, Response, status
from pydantic import BaseModel, Field, field_validator
import uuid

app = FastAPI()

class IngestRequest(BaseModel):
    document_uri: str = Field( ... , max_length=2048)
    tenant_id: str
    priority: str = Field("normal", pattern="^(low|normal|high)$")
    callback_url: str | None = None

    @field_validator("document_uri")
    @classmethod
    def must_be_allowed_scheme(cls, v: str) -> str:
        if not v.startswith(("sharepoint://", "s3://", "https://")):
            raise ValueError("unsupported scheme")
        return v

@app.post("/v1/jobs", status_code=status.HTTP_202_ACCEPTED)
async def create_job(req: IngestRequest, response: Response,
                     idempotency_key: str | None = Header(None,
                         alias="Idempotency-Key")):
    # 1. Validate synchronously — fail fast with a useful message
    if not await tenant_is_active(req.tenant_id):
        raise HTTPException(403, detail={
            "error": "tenant_inactive", "tenant_id": req.tenant_id,
            "remediation": "Contact your administrator to reactivate."})

    # 2. Idempotency — a client retry must not create a second job
    if idempotency_key:
        existing = await jobs.find_by_idempotency_key(req.tenant_id, idempotency_key)
        if existing:
            response.headers["Location"] = f"/v1/jobs/{existing.id}"
            response.status_code = status.HTTP_200_OK
            return existing.as_dict()

    job_id = str(uuid.uuid4())

    # 3. Transactional outbox: the job row and the queue message are written
    #    in ONE database transaction. A separate relay publishes it. This is
    #    the only correct way to avoid "job created but never queued".
    async with db.transaction():
        await jobs.insert(id=job_id, status="queued", request=req.model_dump(),
                          tenant_id=req.tenant_id, idempotency_key=idempotency_key)
        await outbox.insert(topic="doc.ingest.requested", key=job_id,
                            payload={"job_id": job_id, **req.model_dump()})
```

```
    response.headers["Location"] = f"/v1/jobs/{job_id}"
    response.headers["Retry-After"] = "5"            # tell the client how to poll
    return {"job_id": job_id, "status": "queued",
            "status_url": f"/v1/jobs/{job_id}",
            "estimated_seconds": 45}

@app.get("/v1/jobs/{job_id}")
async def get_job(job_id: str):
    job = await jobs.get(job_id)
    if not job:
        raise HTTPException(404)
    body = {"job_id": job.id, "status": job.status,          # queued|running|
            "created_at": job.created_at,                    # succeeded|failed
            "updated_at": job.updated_at,
            "progress": job.progress}
    if job.status == "succeeded":
        body["result"] = job.result
    if job.status == "failed":
        body["error"] = {"code": job.error_code, "message": job.error_message,
                         "retryable": job.retryable,
                         "attempts": job.attempts}
    return body
```

The four interaction patterns

REQUEST / REPLY

Caller waits for a specific answer. Use for queries and for commands whose outcome the caller needs. Over messaging, implement with a correlation ID and a reply-to queue. Always set a timeout and decide what the caller does when it expires — "assume failure" and "assume success" are both wrong without idempotency.

FIRE AND FORGET

Caller sends and continues. Use for notifications, audit events, metrics. The caller gets no confirmation that the work succeeded, so it must not matter — or the channel must be durable enough that the message cannot be lost in transit.

PUBLISH / SUBSCRIBE

One publisher, N subscribers, publisher does not know who they are. Use for domain events — "InvoiceApproved", "EmployeeHired" — where multiple systems need to react and the set of reactors changes over time. This is how you add a consumer without touching the producer, which in an enterprise is worth a great deal politically.

POINT TO POINT

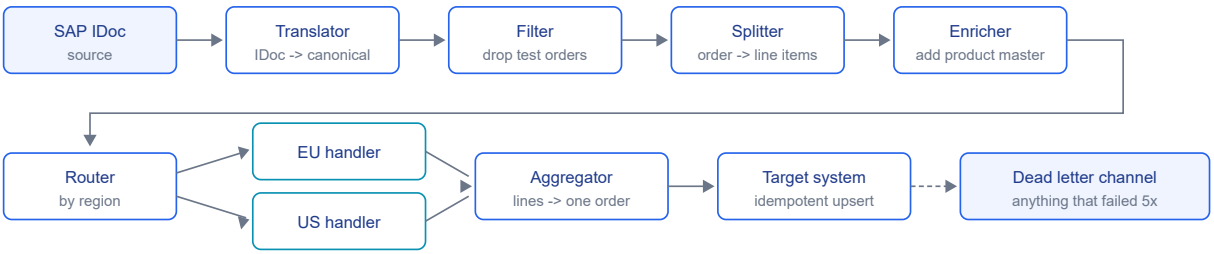
One producer, one logical consumer (possibly many instances, but each message goes to exactly one). Use for work queues where a task must be done once. Competing consumers give you horizontal scaling for free.

> **Events versus commands — a distinction worth enforcing**
> A **command** tells a specific system to do something: `ProcessInvoice`, imperative, one handler, the sender expects it to happen. An **event** states that something already happened: `InvoiceApproved`, past tense, zero or more handlers, the publisher does not care who reacts. Mixing them produces systems where a "publisher" secretly depends on one specific subscriber behaving in one specific way, which is a distributed monolith with extra latency. Name your topics in past tense for events and verb-first for commands, and the design discipline follows from the naming.

The Messaging Patterns Toolkit

Hohpe and Woolf catalogued sixty-five integration patterns in 2003. About a dozen do ninety per cent of the work, and they are as relevant in a Kafka-and-Kubernetes estate as they were in an MQ-and-mainframe one. Learning them gives you a vocabulary: when you say "we need a claim check here" to an enterprise architect, you have just saved twenty minutes of explanation and demonstrated that you have done this before.

A COMPOSED PIPELINE — most real integrations chain five or six patterns



CROSS-CUTTING, APPLIED AT EVERY HOP

Correlation ID on every message | Idempotency key on every write | Message store for replay

Claim check for payloads over ~256 KB | Schema version in the header, never inferred

Dead letter channel with a documented human process | Reconciliation job comparing source and target counts

Without these five, the pipeline above works in demo and fails in month two.

Patterns compose. Name them explicitly in your design document and reviewers recognise them instantly.

The patterns, with modern equivalents

Pattern	What it does	Modern example	Watch out for
Message channel	A named conduit between producer and consumer	Kafka topic, SQS queue, Service Bus topic, RabbitMQ exchange+queue	Channel naming is a public API; rename and you break everyone
Message router	Sends a message to one of several channels based on content	Kafka Streams branch, RabbitMQ topic exchange, EventBridge rules, Camel `choice()`	Routing logic becomes business logic in the wrong place
Message filter	Discards messages that do not match a predicate	Consumer-side predicate, SQS/SNS filter policy, Service Bus SQL filter	Filtering at the consumer still pays the delivery cost; filter at the broker if you can
Message translator	Converts between formats/schemas	Any mapping layer; Camel `Processor`; a dbt model; a Lambda	N systems need N(N−1) translators unless you adopt a canonical model
Content enricher	Adds data the message lacks by looking it up	Join against a cached product master, call a reference API, Kafka Streams KTable join	The lookup is a hidden synchronous dependency; cache it and define staleness
Content filter	Removes fields you do not need or must not carry	Projection before publishing; PII stripping before crossing a boundary	Downstream later needs the field you dropped; version the contract
Splitter	One message becomes many	Order → line items; a 50k-row file → 50k events	Loses the batch boundary; carry a `batch_id` and `index/total`
Aggregator	Many messages become one	Collect all line items of an order; windowed aggregation in Flink/Kafka Streams	Needs a completeness condition AND a timeout; what do you do with 9 of 10?
Scatter-gather	Send to N recipients, combine the replies	Query three pricing engines, take the best;	The slowest recipient sets your latency; set a

		fan-out to regional systems	per-leg deadline and accept partials
Resequencer	Restores order to out-of-order messages	Buffer by sequence number; Kafka ordering within a partition	Unbounded buffering if a message is lost; always bound it
Claim check	Store a large payload, pass a reference	Write to S3/Blob, put the URI in the message	Lifecycle mismatch — the blob is deleted before the message is consumed
Dead letter channel	Where unprocessable messages go	SQS redrive policy, RabbitMQ DLX, Kafka DLQ topic	A DLQ with no owner and no process is a data loss mechanism with a nice name
Idempotent receiver	Processing a duplicate has no additional effect	Dedupe table on message ID, conditional upsert, versioned writes	The dedupe window must exceed the maximum possible redelivery delay
Saga / process manager	Coordinates a long transaction with compensations	Order → reserve stock → charge card → ship, with compensating actions	Compensation is not rollback; a sent email cannot be unsent
Wire tap	Copies messages to a secondary channel	Audit topic, debug sink, a shadow consumer for testing	Doubles your volume and your cost; and the copy contains the same PII
Message store	Durable record of every message for replay and audit	Kafka's log itself; an archive topic; S3 raw landing zone	Retention vs GDPR erasure; plan for it before legal asks

Claim check, worked

PYTHON

```python
# Problem: a 40 MB scanned PDF must flow through a pipeline whose broker has a
# 256 KB message limit (SQS 256KB, Service Bus 256KB standard / 100MB premium,
# Kafka default 1MB). Do not raise the limit. Use a claim check.

import boto3, json, uuid
from datetime import datetime, timedelta, timezone

s3 = boto3.client("s3")
BUCKET = "acme-integration-payloads"

def publish_large(doc_bytes: bytes, meta: dict) -> dict:
    key = f"claims/{datetime.now(timezone.utc):%Y/%m/%d}/{uuid.uuid4()}.pdf"
    s3.put_object(
        Bucket=BUCKET, Key=key, Body=doc_bytes,
        ContentType=meta["mime"],
        ServerSideEncryption="aws:kms",
        SSEKMSKeyId=KMS_KEY_ARN,
        # Lifecycle must OUTLIVE the queue's maximum retention + DLQ retention.
        # Queue 14d + DLQ 14d -> set this to at least 35 days.
        Metadata={"tenant": meta["tenant"], "sha256": meta["sha256"]},
    )
    message = {                                  # small, routable, inspectable
        "schema": "doc.received.v2",
        "claim_check": {"bucket": BUCKET, "key": key,
                        "bytes": len(doc_bytes), "sha256": meta["sha256"]},
        "doc_id": meta["doc_id"],
        "tenant_id": meta["tenant"],
        "mime": meta["mime"],
        "pages": meta["pages"],
        "received_at": datetime.now(timezone.utc).isoformat(),
    }
    queue.send(json.dumps(message),
               attributes={"schema": "doc.received.v2",
                           "trace_id": current_trace_id()})
    return message

def consume(message: dict) -> bytes:
    cc = message["claim_check"]
    try:
        obj = s3.get_object(Bucket=cc["bucket"], Key=cc["key"])
    except s3.exceptions.NoSuchKey:
        # The classic claim-check failure: the blob expired before the message
        # was consumed, usually after a long DLQ sojourn. Make it explicit.
        raise ClaimCheckExpired(
            f"payload {cc['key']} no longer exists; message age "
            f"{message_age(message)}; check the bucket lifecycle policy")
    body = obj["Body"].read()
    # Verify integrity - the message and the blob can drift
```

```
    if hashlib.sha256(body).hexdigest() ≠ cc["sha256"]:
        raise PayloadCorrupt(f"sha256 mismatch for {cc['key']}")
    return body
```

Saga, worked

PYTHON

```python
# A saga for "approve and pay an invoice" across four systems that have no
# shared transaction. Each step has a compensating action. State is persisted
# after every step, because the orchestrator itself can crash.

from enum import Enum

class Step(str, Enum):
    VALIDATE       = "validate"
    RESERVE_BUDGET = "reserve_budget"      # compensate: release_budget
    POST_TO_ERP    = "post_to_erp"         # compensate: reverse_posting
    INITIATE_PAY   = "initiate_payment"    # compensate: cancel_payment (TIME-LIMITED)
    NOTIFY         = "notify"              # compensate: none possible

SAGA = [
    (Step.VALIDATE,       validate_invoice,  None),
    (Step.RESERVE_BUDGET, reserve_budget,    release_budget),
    (Step.POST_TO_ERP,    post_to_erp,       reverse_posting),
    (Step.INITIATE_PAY,   initiate_payment,  cancel_payment),
    (Step.NOTIFY,         notify_requester,  None),
]

def run_saga(saga_id: str, ctx: dict):
    state = store.load(saga_id) or {"completed": [], "ctx": ctx}
    try:
        for step, action, _ in SAGA:
            if step in state["completed"]:
                continue                       # resume after a crash
            # Every action must be idempotent and keyed by (saga_id, step)
            result = action(saga_id, state["ctx"])
            state["ctx"].update(result or {})
            state["completed"].append(step)
            store.save(saga_id, state)         # persist BEFORE the next step
            log.info("saga.step.ok", saga_id=saga_id, step=step)
        store.mark_complete(saga_id)
    except Exception as e:
        log.error("saga.step.failed", saga_id=saga_id,
                  step=step, error=str(e))
        compensate(saga_id, state, failed_at=step)
        raise

def compensate(saga_id: str, state: dict, failed_at: Step):
    """Undo completed steps in REVERSE order. Compensation must be idempotent
    and must itself be retried — a failed compensation is an escalation."""
    for step, _, undo in reversed(SAGA):
        if step not in state["completed"] or undo is None:
            continue
        for attempt in range(1, 6):
            try:
```

```
                undo(saga_id, state["ctx"])
                log.warning("saga.compensated", saga_id=saga_id, step=step)
                break
            except Exception as ce:
                log.error("saga.compensation.failed", saga_id=saga_id,
                          step=step, attempt=attempt, error=str(ce))
                if attempt == 5:
                    # This is the case that needs a human, immediately.
                    alert_oncall("SAGA_COMPENSATION_FAILED",
                                 saga_id=saga_id, step=step,
                                 detail="Manual intervention required: "
                                        "financial state may be inconsistent")
                    store.mark_needs_intervention(saga_id, step)
                    return
                time.sleep(2 ** attempt)
    store.mark_compensated(saga_id)
```

Compensation is not rollback

A database rollback leaves no trace. A compensation is a new business transaction that reverses the effect of a previous one, and it is visible: the ERP shows a posting and a reversal, the customer's bank shows a payment and a refund, the recipient already received the email. Three consequences. One, compensations must be acceptable to the business — get the finance team to agree that a reversal document is fine before you design around it. Two, some steps cannot be compensated at all (an email sent, a physical item shipped), so order your saga to put irreversible steps *last*. Three, compensation can fail, and when it does you have an inconsistent state across systems that only a human can fix — so that path must page someone and must be in the runbook.

Aggregator completeness, the detail people miss

PYTHON

```python
# An aggregator needs THREE things: a correlation key, a completeness
# condition, and a timeout. Omit the timeout and you leak memory forever
# waiting for a message that was lost in 2024.

class OrderAggregator:
    def __init__(self, timeout_s: int = 300):
        self.pending: dict[str, dict] = {}
        self.timeout_s = timeout_s

    def on_line_item(self, msg: dict):
        key = msg["order_id"]                      # 1. correlation key
        entry = self.pending.setdefault(key, {
            "items": {}, "expected": msg.get("total_lines"),
            "first_seen": time.monotonic(), "batch_id": msg.get("batch_id")})
        entry["items"][msg["line_no"]] = msg       # dedupe by line number

        # 2. completeness condition
        if entry["expected"] and len(entry["items"]) == entry["expected"]:
            self._emit(key, entry, complete=True)

    def sweep(self):
        """3. timeout — run every 30s. This is the part that gets forgotten."""
        now = time.monotonic()
        for key, entry in list(self.pending.items()):
            if now - entry["first_seen"] < self.timeout_s:
                continue
            got, want = len(entry["items"]), entry["expected"]
            log.error("aggregator.incomplete", order_id=key,
                      received=got, expected=want,
                      missing=sorted(set(range(1, (want or 0) + 1))
                                     - set(entry["items"])),
                      waited_s=round(now - entry["first_seen"]))
            AGG_INCOMPLETE.labels(reason="timeout").inc()
            # Policy decision, agreed with the business IN ADVANCE:
            #   emit partial  → downstream must tolerate incomplete orders
            #   send to DLQ   → a human investigates
            # Never: silently drop. Never: wait forever.
            self._emit(key, entry, complete=False)

    def _emit(self, key, entry, complete: bool):
        order = build_order(key, entry["items"], complete=complete)
        publisher.send("order.assembled.v1", order,
                       headers={"complete": str(complete).lower()})
        del self.pending[key]
```

Protocols and Formats You Will Actually Meet

You will spend a surprising proportion of your career converting between formats invented by people who never met. The practical skill is not knowing every specification — it is recognising which one you are looking at within ten seconds, knowing its three characteristic failure modes, and knowing which library already solves it so you do not write a parser.

The landscape

Protocol	Transport	Where you meet it	Characteristic pain
REST / JSON	HTTPS	Everything modern; most SaaS	No contract unless OpenAPI exists; pagination styles all differ; "REST" often means RPC over HTTP
OData v2 / v4	HTTPS	SAP Gateway, Dynamics, Microsoft Graph, Business Central	Query syntax is powerful and fiddly; `$expand` depth limits; v2 vs v4 incompatibility; CSRF token dance
SOAP / WSDL	HTTPS, sometimes JMS	Older SAP, Oracle, banking, insurance, government, any pre-2012 enterprise service	XML namespaces, WS-Security headers, MTOM attachments, generated clients that break on a schema change
GraphQL	HTTPS	Modern product APIs, Shopify, GitHub, some internal platforms	N+1 on the server, query cost limits, no HTTP caching, over-fetching by careless clients
gRPC	HTTP/2	Internal microservices, high-throughput internal links	Needs HTTP/2 end to end — enterprise proxies and old load balancers break it; browsers need grpc-web
SFTP / FTPS	SSH / TLS	Banks, logistics, payroll, anything inter-company	Key rotation, directory conventions, partial files, no schema
AS2 / EDI	HTTPS	Retail and logistics EDI, X12 and EDIFACT	MDN receipts, certificate exchange, segment-level spec documents of 400 pages
JMS / AMQP / MQTT	TCP	IBM MQ, ActiveMQ, RabbitMQ, IoT	Broker-specific ports, client library versions, transactional semantics
JDBC / ODBC	TCP	Direct database access to Oracle, DB2, SQL Server	Driver licensing, network path, DBA resistance, long-held connections

RFC / BAPI	SAP proprietary	SAP ECC	Needs the SAP NetWeaver RFC SDK, gateway ports 33xx, Basis team involvement

OData, because you will meet it in every SAP and Microsoft estate

TEXT

```
OData is REST with a standard query language. Learn these eight operators and
you can read any SAP Gateway or Microsoft Graph URL on sight.

  GET /sap/opu/odata/sap/ZSD_ORDER_SRV/OrderSet
      ?$filter=CreatedAt gt datetime'2026-03-01T00:00:00' and Status eq 'OPEN'
      &$select=OrderId,CustomerId,NetAmount,Currency
      &$expand=ToItems
      &$orderby=CreatedAt desc
      &$top=100
      &$skip=0
      &$inlinecount=allpages          (v2)   /  $count=true   (v4)
      &$format=json

  $filter   eq ne gt ge lt le, and or not, substringof/contains, startswith
  $select   projection - ALWAYS use it. Default returns every field, and on an
            SAP order entity that is 180 columns you do not need.
  $expand   join related entities. Depth is usually limited to 1-2 levels and
            SAP Gateway will return 400 if you exceed it.
  $top/$skip  pagination. SAP commonly caps $top at 5000 and the real limit is
            often lower than documented. Test it.
  $orderby  ALWAYS specify a stable sort when paginating, or $skip-based paging
            will silently skip and duplicate rows as data changes underneath you.

V2 vs V4 - the differences that will bite you
  Dates     v2: datetime'2026-03-01T00:00:00'  and /Date(1740787200000)/ in JSON
            v4: 2026-03-01T00:00:00Z (ISO 8601)
  Count     v2: $inlinecount=allpages --> d.__count
            v4: $count=true           --> @odata.count
  Payload   v2: { "d": { "results": [ ... ] } }
            v4: { "value": [ ... ] }
  Next link v2: d.__next        v4: @odata.nextLink
  Errors    v2: error.message.value   v4: error.message (string)

THE SAP CSRF DANCE - required for every POST/PUT/DELETE, trips up everyone
  1. GET any entity with header  x-csrf-token: Fetch
  2. Read the response header    x-csrf-token: <value>  AND keep the cookies
  3. Send the write request with x-csrf-token: <value>  AND the same cookies
  4. On 403 with x-csrf-token: Required --> the token expired, repeat from 1
```

PYTHON

```python
import httpx
from urllib.parse import urljoin

class ODataClient:
    """Handles CSRF, session cookies, paging and v2/v4 shape differences."""

    def __init__(self, base: str, auth, version: str = "v2", verify=None):
        self.base = base.rstrip("/") + "/"
        self.version = version
        self.client = httpx.Client(
            auth=auth, verify=verify,                # verify = path to their CA
            timeout=httpx.Timeout(30.0, connect=10.0),
            headers={"Accept": "application/json"},
            follow_redirects=False,                  # SAP redirects can drop auth
        )
        self._csrf: str | None = None

    def _fetch_csrf(self, entity: str):
        r = self.client.get(urljoin(self.base, entity),
                            params={"$top": 1},
                            headers={"x-csrf-token": "Fetch"})
        r.raise_for_status()
        self._csrf = r.headers.get("x-csrf-token")
        if not self._csrf:
            raise RuntimeError("Gateway did not return a CSRF token. The service "
                               "may not be activated in /IWFND/MAINT_SERVICE.")

    def get_all(self, entity: str, **params):
        """Follow nextLink. NEVER loop on $skip without a stable $orderby."""
        params.setdefault("$format", "json")
        url, first = urljoin(self.base, entity), True
        pages = 0
        while url:
            r = self.client.get(url, params=params if first else None)
            self._raise_for_odata_error(r)
            body = r.json()
            if self.version == "v2":
                d = body["d"]
                items = d.get("results", [d])
                url = d.get("__next")
            else:
                items = body.get("value", [])
                url = body.get("@odata.nextLink")
            yield from items
            first = False
            pages += 1
            if pages > 10_000:                       # guard against an infinite loop
                raise RuntimeError("pagination exceeded 10,000 pages; "
```

```
                                    "the service is probably returning the same "
                                    "nextLink repeatedly")

    def create(self, entity: str, payload: dict, retry_csrf: bool = True):
        if not self._csrf:
            self._fetch_csrf(entity)
        r = self.client.post(urljoin(self.base, entity), json=payload,
                             headers={"x-csrf-token": self._csrf,
                                      "Content-Type": "application/json"})
        if r.status_code == 403 and \
           r.headers.get("x-csrf-token", "").lower() == "required" and retry_csrf:
            self._csrf = None
            return self.create(entity, payload, retry_csrf=False)
        self._raise_for_odata_error(r)
        return r.json()

    @staticmethod
    def _raise_for_odata_error(r: httpx.Response):
        if r.is_success:
            return
        # SAP returns rich errors. Surface them — "400 Bad Request" alone is useless.
        try:
            err = r.json()["error"]
            msg = err.get("message")
            msg = msg.get("value") if isinstance(msg, dict) else msg
            code = err.get("code")
            details = err.get("innererror", {}).get("errordetails", [])
            raise ODataError(f"{r.status_code} {code}: {msg}; "
                             f"details={[d.get('message') for d in details][:3]}")
        except (ValueError, KeyError):
            raise ODataError(f"{r.status_code}: {r.text[:500]}")
```

> ⚠ **$skip-based pagination is unsafe on live data**
>
> If you page with `$top=1000&$skip=0`, then `$skip=1000`, and rows are inserted or deleted between the two calls, you will skip records and duplicate others — silently. The reconciliation job catches it three weeks later and nobody can explain it. Always either follow the server's `nextLink` /skiptoken (which uses a stable cursor), or page on a monotonic key: `$filter=Id gt '<lastId>'&$orderby=Id&$top=1000`. The second approach also makes your job resumable after a crash, which matters for an eight-hour initial load.

SOAP, which is not dead

```xml
```

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
                  xmlns:urn="urn:sap-com:document:sap:soap:functions:mc-style">
  <soapenv:Header>
    <wsse:Security xmlns:wsse="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss
                   soapenv:mustUnderstand="1">
      <wsse:UsernameToken>
        <wsse:Username>SVC_INTEGRATION</wsse:Username>
        <wsse:Password Type=" ... #PasswordText">${SECRET}</wsse:Password>
      </wsse:UsernameToken>
    </wsse:Security>
  </soapenv:Header>
  <soapenv:Body>
    <urn:SalesOrderCreateFromDat2>
      <OrderHeaderIn>
        <DocType>TA</DocType>
        <SalesOrg>1000</SalesOrg>
        <DistrChan>10</DistrChan>
        <Division>00</Division>
      </OrderHeaderIn>
      <OrderItemsIn>
        <item>
          <ItmNumber>000010</ItmNumber>
          <Material>000000000000012345</Material>
          <ReqQty>10.000</ReqQty>
        </item>
      </OrderItemsIn>
    </urn:SalesOrderCreateFromDat2>
  </soapenv:Body>
</soapenv:Envelope>
```

PYTHON

```
# zeep is the only Python SOAP client worth using.
from zeep import Client, Settings
from zeep.transports import Transport
from zeep.wsse.username import UsernameToken
from requests import Session
from requests.auth import HTTPBasicAuth

session = Session()
session.verify = "/etc/ssl/acme-internal-ca.pem"   # their private CA
session.auth = HTTPBasicAuth(user, password)       # some services want BOTH
session.proxies = {"https": "http://proxy.acme.corp:8080"}

client = Client(
    wsdl="https://sap-pi.acme.corp/dir/wsdl?p=ic/SalesOrderCreate",
    transport=Transport(session=session, timeout=30, operation_timeout=60,
                        cache=SqliteCache(path="/tmp/zeep.db", timeout=3600)),
    wsse=UsernameToken(user, password),
    settings=Settings(strict=False,              # tolerate minor schema drift
                      xml_huge_tree=True,        # SAP responses can be enormous
                      raw_response=False),
)

# ALWAYS do this first and commit the output to the repo. The WSDL will change
# without warning and you want a diff, not a 3am surprise.
#   python -m zeep https://... /wsdl > docs/salesorder.wsdl.txt

try:
    result = client.service.SalesOrderCreateFromDat2(
        OrderHeaderIn={"DocType": "TA", "SalesOrg": "1000",
                       "DistrChan": "10", "Division": "00"},
        OrderItemsIn={"item": [{"ItmNumber": "000010",
                                "Material": "000000000000012345",
                                "ReqQty": "10.000"}]},
    )
except Fault as f:
    # SOAP faults carry structured detail — log it, do not swallow it
    log.error("soap.fault", code=f.code, message=f.message,
              detail=etree.tostring(f.detail).decode()[:2000] if f.detail else None)
    raise

# SAP BAPIs return errors INSIDE a successful SOAP response. A 200 does not
# mean success. This check is mandatory and is the most common omission.
messages = result.Return.item if result.Return else []
errors = [m for m in messages if m.Type in ("E", "A")]   # E=error, A=abort
if errors:
    raise BapiError("; ".join(f"{m.Id}/{m.Number}: {m.Message}" for m in errors))
warnings = [m for m in messages if m.Type == "W"]
```

```
if warnings:
    log.warning("bapi.warnings", messages=[m.Message for m in warnings])
```

> ✗ **HTTP 200 does not mean success**
> This is the single most common integration bug in enterprise systems. SAP BAPIs return a `RETURN` table of messages with a type column; a successful HTTP call can contain fatal errors. Salesforce's composite and bulk APIs return 200 with a per-record success flag. Many older SOAP services return a `<ResultCode>` element. Several ESBs return 200 with a fault payload. Always inspect the body, define what success means for that specific service, and write that definition into the integration specification so the next engineer does not rediscover it.

Data formats and their traps

Format	Seen in	Trap
JSON	Everywhere	No native date type; numbers lose precision above 2^53 (SAP document numbers and Twitter-style IDs); no comments; duplicate keys undefined
XML	SOAP, IDoc, EDI envelopes, config	Namespaces; entity expansion (billion laughs) and XXE — **always disable external entities**; mixed content; whitespace significance
CSV	File drops, exports, Excel	Delimiter inside a quoted field; embedded newlines; BOM; encoding; no type information; Excel mangling leading zeros and dates
Fixed width	Mainframe, banking, payroll	Column positions in a PDF spec; EBCDIC; signed overpunch numerics; padding rules
EDIFACT / X12	Logistics, retail, healthcare	Segment terminators, release characters, 400-page implementation guides, partner-specific variants
SAP IDoc	SAP to anything	Flat or XML form; segment definitions versioned by SAP release; control record semantics
Parquet / Avro	Data platform	Schema evolution rules; logical type mismatches between writers and readers
Excel (.xlsx)	Business users; more often than you expect	Merged cells, formulas vs values, multiple sheets, a human editing it while you read it

PYTHON

```python
# XML parsing in an enterprise context — the secure baseline.
# lxml and the stdlib are both vulnerable by default to XXE and entity bombs.
from lxml import etree

parser = etree.XMLParser(
    resolve_entities=False,   # blocks XXE: <!ENTITY x SYSTEM "file:///etc/passwd">
    no_network=True,          # blocks fetching remote DTDs
    huge_tree=False,          # blocks billion-laughs memory exhaustion
    recover=False,            # fail loudly on malformed input, do not guess
)
root = etree.fromstring(payload_bytes, parser)

# Namespaces: the #1 reason an XPath "does not work"
NS = {"soap": "http://schemas.xmlsoap.org/soap/envelope/",
      "urn":  "urn:sap-com:document:sap:soap:functions:mc-style"}
orders = root.findall(".//urn:Order", NS)     # without NS this returns []

# Streaming for large documents — a 2 GB IDoc file will not fit in memory
for event, elem in etree.iterparse(path, events=("end",), tag="IDOC",
                                   huge_tree=False):
    handle_idoc(elem)
    elem.clear()                                  # free the subtree
    while elem.getprevious() is not None:         # free preceding siblings too
        del elem.getparent()[0]
```

PYTHON

```python
# Fixed-width and EBCDIC — mainframe extracts, still extremely common
from dataclasses import dataclass
from decimal import Decimal

# The spec arrives as a PDF table. Encode it as data, not as slice literals
# scattered through the code.
LAYOUT = [
    # (name,          start, length, type)      1-based positions from the spec
    ("record_type",     1,  2, "A"),
    ("account_no",      3, 10, "N"),
    ("customer_name",  13, 35, "A"),
    ("balance",        48, 13, "S2"),    # signed, 2 implied decimal places
    ("currency",       61,  3, "A"),
    ("open_date",      64,  8, "D"),     # YYYYMMDD
    ("filler",         72,  8, "A"),
]

def parse_signed_overpunch(raw: str) -> Decimal:
    """Mainframe zoned-decimal: the sign is encoded in the LAST character.
    '1234J' = -12341  ... this is not a joke and you will meet it."""
    POS = {"{": "0", "A": "1", "B": "2", "C": "3", "D": "4",
           "E": "5", "F": "6", "G": "7", "H": "8", "I": "9"}
    NEG = {"}": "0", "J": "1", "K": "2", "L": "3", "M": "4",
           "N": "5", "O": "6", "P": "7", "Q": "8", "R": "9"}
    body, last = raw[:-1], raw[-1]
    if last in POS:  return Decimal(body + POS[last])
    if last in NEG:  return Decimal("-" + body + NEG[last])
    return Decimal(raw)

def parse_line(line: str) -> dict:
    out = {}
    for name, start, length, kind in LAYOUT:
        raw = line[start - 1: start - 1 + length]
        if   kind == "A":  out[name] = raw.strip()
        elif kind == "N":  out[name] = int(raw or 0)
        elif kind == "D":  out[name] = (datetime.strptime(raw, "%Y%m%d").date()
                                        if raw.strip() and raw ≠ "00000000" else None
        elif kind.startswith("S"):
            dp = int(kind[1:])
            out[name] = parse_signed_overpunch(raw.strip()) / (10 ** dp)
    return out

# EBCDIC: mainframe files are often cp037 (US) or cp500 (international).
# A file that looks like binary garbage in a text editor is usually EBCDIC.
with open(path, "r", encoding="cp037", newline="") as fh:
    for line in fh:
        record = parse_line(line.rstrip("\n"))
```

Webhooks

A webhook is an HTTP callback: when something happens in their system, they POST to your URL. It is the cheapest way to get near-real-time events out of a SaaS platform, and it is also the integration most likely to lose data quietly, because the sender's idea of "delivered" is a 200 response and nothing else. You will both consume webhooks and build them, and the engineering is asymmetric.

Consuming webhooks safely

1. **Verify the signature before anything else.**
   Your endpoint is on the public internet, or at least reachable by whatever can reach your gateway. Anyone can POST to it. Use HMAC verification with a constant-time comparison, against the *raw* request body, before parsing.

2. **Check the timestamp.**
   A valid signature on a replayed payload from last Tuesday is still valid. Reject anything outside a five-minute window and keep a short-lived cache of seen event IDs.

3. **Return 200 fast and do the work later.**
   Most providers time out at 3–10 seconds and will retry or disable your endpoint if you are slow. Validate, persist to a queue, return. Never do business logic inside the request.

4. **Be idempotent.**
   Every provider will deliver the same event twice eventually — at-least-once is the industry norm. Deduplicate on the provider's event ID.

5. **Do not trust the payload's contents.**
   Many providers deliberately send thin payloads. Even when the payload is fat, treat it as a hint and fetch the current state from the API, because webhooks arrive out of order and a stale update can overwrite a newer one.

6. **Handle out-of-order delivery.**
   Compare a version or `updated_at` field and discard anything older than what you already have.

7. **Have a reconciliation path.**
   Webhooks will be missed — the provider had an outage, your endpoint was down past their retry limit, a firewall rule changed. A periodic poll of "everything changed since T" is not optional; it is the safety net.

PYTHON

```python
import hmac, hashlib, time, json
from fastapi import APIRouter, Request, Response, HTTPException

router = APIRouter()
TOLERANCE_S = 300

@router.post("/webhooks/crm")
async def receive(request: Request):
    raw = await request.body()          # RAW bytes. Never re-serialise the parsed
                                        # JSON before verifying — key order changes
                                        # and the signature will never match.
    sig_header = request.headers.get("X-Signature", "")
    ts_header  = request.headers.get("X-Timestamp", "")

    # --- 1. Replay window ------------------------------------------------
    try:
        ts = int(ts_header)
    except ValueError:
        raise HTTPException(400, "missing or invalid timestamp")
    if abs(time.time() - ts) > TOLERANCE_S:
        WEBHOOK_REJECTED.labels(reason="stale").inc()
        raise HTTPException(400, "timestamp outside tolerance")

    # --- 2. Signature over timestamp + body (prevents detaching the body) --
    signed_payload = f"{ts}.".encode() + raw
    expected = hmac.new(WEBHOOK_SECRET, signed_payload, hashlib.sha256).hexdigest()
    provided = sig_header.removeprefix("sha256=")
    if not hmac.compare_digest(expected, provided):      # constant time
        WEBHOOK_REJECTED.labels(reason="bad_signature").inc()
        log.warning("webhook.bad_signature",
                    remote=request.client.host, bytes=len(raw))
        raise HTTPException(401, "invalid signature")

    # --- 3. Parse only AFTER verification --------------------------------
    try:
        event = json.loads(raw)
    except json.JSONDecodeError:
        raise HTTPException(400, "invalid JSON")

    event_id = event.get("id")
    if not event_id:
        raise HTTPException(400, "missing event id")

    # --- 4. Idempotency. SETNX with a TTL longer than the provider's
    #        maximum retry window (Stripe: 3 days; most: 24h).
    if not await redis.set(f"wh:crm:{event_id}", "1", nx=True, ex=4 * 86400):
        WEBHOOK_DUPLICATE.inc()
        return Response(status_code=200)      # 200, not 409 — it IS handled
```

```
    # --- 5. Persist and return. Work happens in a consumer.
    await outbox.insert(topic="crm.event.received", key=event_id, payload=event)
    WEBHOOK_ACCEPTED.labels(type=event.get("type", "unknown")).inc()
    return Response(status_code=200)          # under 100ms, every time
```

⚠ **Signature schemes differ; read their docs, do not guess**

Stripe signs `timestamp.body` with HMAC-SHA256 and sends `t=…,v1=…` in one header. GitHub signs the raw body and sends `sha256=…` in `X-Hub-Signature-256`. Slack signs `v0:timestamp:body`. Shopify base64-encodes the HMAC rather than hex-encoding it. Twilio signs the URL plus sorted POST parameters. Microsoft Graph uses a validation token handshake on subscription creation and a `clientState` value thereafter. Getting this wrong fails closed, which is at least safe — but you will spend an afternoon on it, so budget for that and write an integration test with a captured real payload.

Building webhooks others consume

PYTHON

```python
# If you emit webhooks into a customer's estate, you are now responsible for
# delivery guarantees. Build these six things or do not offer webhooks.

import httpx, hmac, hashlib, time, json, random
from dataclasses import dataclass

RETRY_SCHEDULE_S = [0, 10, 60, 300, 1800, 7200, 21600]   # 7 attempts over ~8h

@dataclass
class Subscription:
    id: str
    url: str
    secret: bytes
    event_types: set[str]
    active: bool
    consecutive_failures: int

async def deliver(sub: Subscription, event: dict, attempt: int = 0):
    body = json.dumps(event, separators=(",", ":"), sort_keys=True).encode()
    ts = int(time.time())
    sig = hmac.new(sub.secret, f"{ts}.".encode() + body, hashlib.sha256).hexdigest()

    headers = {
        "Content-Type": "application/json",
        "X-Timestamp": str(ts),
        "X-Signature": f"sha256={sig}",
        "X-Event-Id": event["id"],
        "X-Event-Type": event["type"],
        "X-Delivery-Attempt": str(attempt + 1),
        "X-Webhook-Version": "2026-01-15",       # date-versioned payload schema
        "User-Agent": "DocPipeline-Webhooks/2.7 (+https://docs.example/webhooks)",
    }
    try:
        async with httpx.AsyncClient(timeout=10.0) as c:
            r = await c.post(sub.url, content=body, headers=headers)
    except httpx.RequestError as e:
        return await _schedule_retry(sub, event, attempt, reason=type(e).__name__)

    await delivery_log.insert(subscription_id=sub.id, event_id=event["id"],
                              attempt=attempt + 1, status=r.status_code,
                              duration_ms=r.elapsed.total_seconds() * 1000,
                              response_excerpt=r.text[:500])

    if 200 <= r.status_code < 300:
        await subs.reset_failures(sub.id)
        return True
    if r.status_code in (400, 401, 403, 404, 410, 422):
        # Permanent: retrying will never help. 410 Gone means unsubscribe.
```

```
        await subs.record_permanent_failure(sub.id, r.status_code)
        if r.status_code == 410:
            await subs.deactivate(sub.id, reason="endpoint_gone")
        return False
    return await _schedule_retry(sub, event, attempt, reason=str(r.status_code))

async def _schedule_retry(sub, event, attempt, reason):
    if attempt + 1 ≥ len(RETRY_SCHEDULE_S):
        await dlq.insert(subscription_id=sub.id, event=event, reason=reason)
        await subs.increment_failures(sub.id)
        # Auto-disable after sustained failure, and TELL the customer — an
        # endpoint that has been silently disabled for a month is a disaster.
        if sub.consecutive_failures ≥ 50:
            await subs.deactivate(sub.id, reason="too_many_failures")
            await notify_customer_admin(sub, "Your webhook endpoint has been "
                                             "disabled after 50 consecutive "
                                             "failures. Events are retained "
                                             "for 7 days and can be replayed.")
        return False
    delay = RETRY_SCHEDULE_S[attempt + 1] * random.uniform(0.8, 1.2)   # jitter
    await scheduler.enqueue_at(time.time() + delay, deliver, sub, event, attempt + 1)
    return False
```

ALSO PROVIDE A REPLAY API

`POST /webhooks/replay?from=…&to=…&types=…`. Customers' endpoints go down; without replay their only recovery is a full reconciliation, which they will ask you to run manually at 6pm on a Friday.

ALSO PROVIDE A DELIVERY LOG

Let the customer see, per event: attempts, status codes, response bodies and timings. This converts "your webhooks are broken" into "our endpoint returned 502 nine times" in one screenshot, and they can self-serve.

VERSION THE PAYLOAD

Date-version the schema (`2026-01-15`), send it in a header, and never remove a field within a version. Additive changes only. Give twelve months of notice for a version retirement.

PUBLISH YOUR SOURCE IPS

Enterprise customers will need to allow-list you. Publish a stable set of egress IPs or a CIDR, commit to notice before changing them, and expect this to be the first question their network team asks.

Message Brokers and Event Streaming

You will not usually choose the broker. The customer has one — often three, inherited from different eras — and your job is to use it correctly. But you must understand the differences, because the delivery semantics, ordering guarantees and failure modes vary enormously, and designing for Kafka semantics on top of a queue (or vice versa) produces subtle data loss.

Queue versus log

TEXT

```
QUEUE (RabbitMQ, ActiveMQ, IBM MQ, SQS, Azure Service Bus)
  - A message is CONSUMED and removed. One logical consumer per message.
  - The broker tracks per-message state: unacked, acked, dead-lettered.
  - Redelivery on nack or on consumer death. Visibility timeouts.
  - Rich routing: headers, topics, priorities, TTL, scheduled delivery.
  - Great for: work distribution, task queues, RPC over messaging, commands.
  - Weak at: replay (the message is gone), many independent consumers,
    very high throughput, retaining history.

LOG (Kafka, Pulsar, Kinesis, Event Hubs, Redpanda)
  - A message is APPENDED to an immutable, ordered, partitioned log.
  - Consumers track their own OFFSET. Reading does not remove anything.
  - Any number of independent consumer groups read the same data.
  - Replay is trivial: reset the offset to the beginning.
  - Great for: event streaming, multiple consumers, replay, audit, very high
    throughput, stream processing, rebuilding a downstream store from scratch.
  - Weak at: per-message retry/DLQ semantics (you build them), selective
    acknowledgement, priority, very large messages, low-volume fan-out
    where a queue would be simpler.

THE TEST
  "Will more than one independent system need this data, now or later?"   → log
  "Does each item need to be worked on exactly once by one worker?"       → queue
  "Will I need to reprocess history after fixing a bug?"                  → log
  "Do I need per-message retry, delay and dead-lettering out of the box?" → queue
```

	RabbitMQ	Kafka	SQS	Azure Service Bus	IB
Model	Queue + exchange routing	Partitioned log	Queue	Queue + topic	Q
Ordering	Per queue, single consumer	Per partition	FIFO queues only	Sessions	Pe
Replay	No	Yes, by offset	No	No (peek-lock only)	N
Retention	Until consumed	Time or size based	Up to 14 days	Up to 14 days	U
Max message	~128 MB (don't)	1 MB default	256 KB	256 KB / 100 MB premium	1(
DLQ	Dead letter exchange	Build it yourself	Redrive policy	Built in	Ba
Throughput	Tens of thousands/s	Millions/s	Effectively unlimited	High	M
Enterprise presence	Very common	Common and growing	AWS shops	Azure shops	Ba m es

Kafka concepts that determine your design

TEXT

```
TOPIC → PARTITIONS → OFFSETS

  topic "invoice.approved.v1", 6 partitions
    P0: [0][1][2][3][4]...        consumer-group "erp-sync"   offset 3
    P1: [0][1][2]...              consumer-group "analytics"  offset 0
     ...

  PARTITION COUNT is the unit of parallelism. A consumer group can have at
  most as many ACTIVE consumers as there are partitions. 6 partitions means
  a maximum of 6 parallel consumers; the 7th sits idle. You can increase
  partitions later but NOT decrease, and increasing breaks key-ordering for
  existing keys because the hash-to-partition mapping changes.

  KEY determines the partition: partition = hash(key) % num_partitions.
    key = invoice_id   → all events for one invoice are ordered, spread evenly
    key = tenant_id    → ordered per tenant, but one huge tenant creates a
                         HOT PARTITION and that consumer falls behind alone
    key = null         → round robin, no ordering guarantee at all

  ORDERING is guaranteed ONLY within a partition. There is no global order.
  If your design needs global ordering you need one partition, which means
  one consumer, which means no horizontal scaling. Usually the right answer
  is to find a key that gives you the ordering you actually need.

  CONSUMER GROUPS are independent. Adding a new group does not affect existing
  ones and starts from the configured offset. This is the property that makes
  Kafka politically useful in an enterprise: you can add a consumer without
  asking the producing team for anything.

  REBALANCING happens when a consumer joins, leaves or times out. During a
  rebalance, consumption STOPS for the group. Long processing times trigger
  max.poll.interval.ms and cause a rebalance storm:
      consumer takes 6 min to process a batch
      → exceeds max.poll.interval.ms (default 5 min)
      → broker declares it dead, rebalances
      → another consumer gets the same messages, also takes 6 min
      → permanent rebalancing, zero progress, growing lag
  FIX: smaller max.poll.records, longer max.poll.interval.ms, or hand work to
  a thread pool and pause()/resume() the partition.
```

PYTHON

```python
from confluent_kafka import Consumer, Producer, KafkaError, TopicPartition
import json

# ———————— Producer: configured for durability, not for speed ————————————
producer = Producer({
    "bootstrap.servers": "kafka-1.acme.corp:9093,kafka-2.acme.corp:9093",
    "security.protocol": "SASL_SSL",
    "sasl.mechanisms": "SCRAM-SHA-512",
    "sasl.username": "svc_docpipeline",
    "sasl.password": SECRET,
    "ssl.ca.location": "/etc/ssl/acme-internal-ca.pem",

    "acks": "all",                      # wait for all in-sync replicas. NEVER use 1
                                        # in an enterprise flow carrying financial dat
    "enable.idempotence": True,         # dedupes producer retries; requires acks=all
    "max.in.flight.requests.per.connection": 5,   # ≤5 keeps ordering with idempotenc
    "retries": 2147483647,
    "delivery.timeout.ms": 120000,      # the real bound on how long a send can take
    "compression.type": "zstd",         # 3-5x reduction on JSON; big WAN saving
    "linger.ms": 20,                    # batch for 20ms — large throughput win
    "batch.size": 131072,
})

def on_delivery(err, msg):
    if err:
        # This is the ONLY place you learn a send failed. Without this callback
        # produce() is fire-and-forget and you will lose messages silently.
        log.error("kafka.delivery_failed", topic=msg.topic(), key=msg.key(),
                  error=str(err))
        FAILED_PRODUCE.labels(msg.topic()).inc()
        dead_letter_locally(msg)
    else:
        log.debug("kafka.delivered", topic=msg.topic(),
                  partition=msg.partition(), offset=msg.offset())

producer.produce(
    topic="invoice.approved.v1",
    key=invoice_id.encode(),            # ordering key
    value=json.dumps(event).encode(),
    headers=[("schema", b"invoice.approved.v1"),
             ("traceparent", current_traceparent().encode()),
             ("source", b"docpipeline")],
    on_delivery=on_delivery,
)
producer.poll(0)                        # serve delivery callbacks
# producer.flush(30) before shutdown, ALWAYS, or in-flight messages are lost

# ———————— Consumer: manual commit, after successful processing ————————————
```

```
consumer = Consumer({
    "bootstrap.servers": "kafka-1.acme.corp:9093",
    "group.id": "docpipeline-erp-sync",
    "enable.auto.commit": False,        # commit AFTER work, not before
    "auto.offset.reset": "earliest",    # new group reads from the beginning
    "max.poll.interval.ms": 600000,     # 10 min — must exceed worst-case batch time
    "session.timeout.ms": 45000,
    "max.poll.records": 100,            # keep batches small enough to finish
    "isolation.level": "read_committed",
})
consumer.subscribe(["invoice.approved.v1"])

while running:
    msgs = consumer.consume(num_messages=100, timeout=1.0)
    for msg in msgs:
        if msg.error():
            if msg.error().code() == KafkaError._PARTITION_EOF:
                continue
            log.error("kafka.consume_error", error=str(msg.error()))
            continue
        try:
            headers = dict(msg.headers() or [])
            ctx = extract_trace_context(headers)
            with tracer.start_as_current_span("invoice.sync", context=ctx):
                process(json.loads(msg.value()))        # MUST be idempotent
        except PoisonMessage as e:
            # Non-retryable: route to a DLQ topic and move on, or you block
            # the whole partition forever.
            producer.produce("invoice.approved.v1.dlq", key=msg.key(),
                             value=msg.value(),
                             headers=(msg.headers() or []) + [
                                 ("dlq_reason", str(e).encode()[:900]),
                                 ("dlq_at", str(int(time.time())).encode()),
                                 ("original_partition", str(msg.partition()).encode()),
                                 ("original_offset", str(msg.offset()).encode())])
            producer.poll(0)
        except TransientError:
            # Retryable: do NOT commit. Seek back and let the next poll retry.
            consumer.seek(TopicPartition(msg.topic(), msg.partition(), msg.offset()))
            time.sleep(backoff())
            break
    if msgs:
        consumer.commit(asynchronous=False)             # commit the batch
# Always close cleanly so the group rebalances promptly instead of waiting
# for session.timeout.ms.
consumer.close()
```

Exactly-once is mostly a marketing term

Kafka's exactly-once semantics are real but narrow: they apply to a read-process-write cycle entirely within Kafka, using transactions, with `isolation.level=read_committed`. The moment your consumer writes to an external system — a database, SAP, an HTTP API — you are back to at-least-once, because there is no distributed transaction across Kafka and SAP. The correct engineering position is: **assume at-least-once delivery and make every consumer idempotent.** That is achievable, testable and explainable to a customer. Promising exactly-once is a promise you cannot keep, and the day a network blip causes a duplicate posting in their general ledger is the day that promise is examined.

The transactional outbox

The dual-write problem: you need to update your database and publish an event. If you write the database then publish, a crash in between loses the event. If you publish then write, a crash loses the database change while the world has been told it happened. There is no ordering that fixes it, because the two systems have no shared transaction. The outbox pattern solves it by making the publish part of the database transaction.

SQL

```sql
-- 1. The outbox table lives in the SAME database as your business data
CREATE TABLE outbox (
  id             BIGSERIAL PRIMARY KEY,
  aggregate_type TEXT        NOT NULL,     -- 'invoice'
  aggregate_id   TEXT        NOT NULL,     -- '4411-2026'
  event_type     TEXT        NOT NULL,     -- 'invoice.approved.v1'
  payload        JSONB       NOT NULL,
  headers        JSONB       NOT NULL DEFAULT '{}',
  created_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
  published_at   TIMESTAMPTZ,
  attempts       INT         NOT NULL DEFAULT 0,
  last_error     TEXT
);
CREATE INDEX outbox_unpublished ON outbox (id) WHERE published_at IS NULL;

-- 2. Business write and event in ONE transaction. Atomic by construction.
BEGIN;
  UPDATE invoices SET status = 'approved', approved_at = now(),
         approved_by = $2 WHERE id = $1;
  INSERT INTO outbox (aggregate_type, aggregate_id, event_type, payload, headers)
  VALUES ('invoice', $1, 'invoice.approved.v1',
          jsonb_build_object('invoice_id', $1, 'approved_by', $2,
                             'amount_minor', $3, 'currency', $4,
                             'occurred_at', now()),
          jsonb_build_object('traceparent', $5, 'schema', 'invoice.approved.v1'));
COMMIT;
```

PYTHON

```python
# 3. A relay publishes committed rows. Two implementations:
#    (a) polling — simple, portable, works anywhere, ~1s latency
#    (b) CDC via Debezium reading the WAL — no polling, lower latency, more moving par

def relay_tick(batch: int = 500):
    with db.transaction():
        rows = db.query("""
            SELECT id, event_type, aggregate_id, payload, headers
            FROM outbox
            WHERE published_at IS NULL
            ORDER BY id                        -- preserves per-aggregate order
            LIMIT %s
            FOR UPDATE SKIP LOCKED             -- lets N relay instances run safely
        """, (batch,))

        for r in rows:
            try:
                producer.produce(
                    topic=r["event_type"],
                    key=r["aggregate_id"].encode(),
                    value=json.dumps(r["payload"]).encode(),
                    headers=[(k, str(v).encode()) for k, v in r["headers"].items()]
                            + [("outbox_id", str(r["id"]).encode())],
                )
            except BufferError:
                producer.flush(5)
                raise                          # retry the whole batch next tick

        producer.flush(30)                     # block until brokers acked ALL
        db.execute("UPDATE outbox SET published_at = now() WHERE id = ANY(%s)",
                   ([r["id"] for r in rows],))

# NOTE: this is at-least-once. If the process dies after flush() but before the
# UPDATE, the batch is republished. That is correct and expected — consumers
# are idempotent. The outbox guarantees NO LOST EVENTS, not no duplicates.

# 4. Housekeeping — the step everyone forgets until the table is 400 GB
#    DELETE FROM outbox WHERE published_at < now() - interval '7 days';
#    Run it in batches with a LIMIT, or the DELETE locks the table.
```

✦ **Consumer lag is the health metric for any streaming integration**
Lag is the difference between the latest offset in a partition and the consumer group's committed offset — how far behind you are, in messages. Expose it (`kafka_consumergroup_lag` from `kafka_exporter` , or Burrow), graph it per partition, and alert on *sustained growth* rather than absolute value, because the absolute number is meaningless without the processing rate. Also alert on **lag in seconds** rather than messages where you can — "we are 40 minutes behind" is a sentence the business understands and "we are 180,000 messages behind" is not. A single hot partition with growing lag while the others are at zero is the classic symptom of a bad partition key.

Reliability Engineering for Integrations

An integration that works is easy. An integration that keeps working for three years without you, across network blips, upstream outages, credential rotations, schema drift and a datacentre migration, is an engineering discipline with a small number of named techniques. This section is the core of the craft.

Delivery semantics

Guarantee	Means	Achieved by	Use when
At-most-once	Message delivered zero or one times. May be lost.	Fire and forget; commit offset before processing	Metrics, telemetry, non-critical notifications
At-least-once	Delivered one or more times. Never lost, may duplicate.	Ack after processing; retry on failure	Almost everything. This is the default and the right default.
Effectively-once	At-least-once delivery + idempotent processing	Dedupe store, conditional writes, natural idempotence	Financial postings, inventory movements, anything with side effects
Exactly-once	Delivered and processed precisely once	Only within a single transactional system	Do not promise this across system boundaries

Idempotency, the four implementations

PYTHON

```
# 1. NATURAL IDEMPOTENCE — the best option when the data model allows it.
#    An UPSERT keyed on a business key. Running it twice changes nothing.
db.execute("""
    INSERT INTO invoices (invoice_id, vendor_id, amount_minor, currency, status,
                          source_version, updated_at)
    VALUES (%(invoice_id)s, %(vendor_id)s, %(amount)s, %(ccy)s, %(status)s,
            %(version)s, now())
    ON CONFLICT (invoice_id) DO UPDATE SET
        amount_minor   = EXCLUDED.amount_minor,
        status         = EXCLUDED.status,
        source_version = EXCLUDED.source_version,
        updated_at     = now()
    -- LAST-WRITE-WINS IS WRONG for out-of-order delivery. Guard with a version:
    WHERE invoices.source_version < EXCLUDED.source_version
""", params)

# 2. DEDUPE STORE — when the operation is not naturally idempotent
#    (sending an email, calling a non-idempotent API, incrementing a balance)
def handle_once(message_id: str, ttl_days: int = 7):
    acquired = redis.set(f"dedupe:{message_id}", "processing",
                         nx=True, ex=ttl_days * 86400)
    if not acquired:
        state = redis.get(f"dedupe:{message_id}")
        if state == b"processing":
            # A concurrent or crashed attempt. Do NOT process; let it retry later.
            raise InProgress(message_id)
        DUPLICATE_SUPPRESSED.inc()
        return False
    return True
# TTL must exceed the maximum possible redelivery window: broker retention +
# DLQ retention + any manual replay window. 7 days is a common safe floor.

# 3. IDEMPOTENCY KEY ON AN OUTBOUND CALL — the client-side equivalent
resp = http.post("https://api.vendor.com/v1/payments",
                 json=payload,
                 headers={"Idempotency-Key": f"pay-{invoice_id}-{attempt_group}"})
# The key must be stable across retries of the SAME logical operation and
# different across different operations. Derive it from business identifiers,
# NEVER from uuid4() generated at call time — that defeats the entire purpose.

# 4. CONDITIONAL WRITE / OPTIMISTIC CONCURRENCY — safest for state machines
db.execute("""
    UPDATE invoices SET status = 'paid', paid_at = now(), version = version + 1
    WHERE invoice_id = %s AND status = 'approved' AND version = %s
""", (invoice_id, expected_version))
if cursor.rowcount == 0:
    # Either someone else changed it, or we already did this. Both are safe.
    current = db.fetch_one("SELECT status, version FROM invoices WHERE invoice_id=%s",
```

```
                            (invoice_id,))
    if current["status"] == "paid":
        log.info("already_applied", invoice_id=invoice_id)   # idempotent success
        return
    raise ConcurrentModification(invoice_id, expected_version, current["version"])
```

> ⚠ **Idempotency has a scope and a window — state both**
> "This endpoint is idempotent" is incomplete. Idempotent for how long? Scoped to which tenant? Keyed on what? A payment API that dedupes for 24 hours is not idempotent against a replay you run after a three-day outage. Write it into the integration specification: "Idempotency keys are honoured for 7 days, scoped per tenant, and a repeated key with a different payload returns 409." That last clause matters — silently returning the original result for a *different* payload hides a client bug.

Retries that help instead of hurt

TEXT

```
WHAT TO RETRY
  Retryable:     429, 500, 502, 503, 504, connection reset, connection timeout,
                 read timeout, DNS temporary failure, broker not available
  NOT retryable: 400, 401, 403, 404, 409 (usually), 422, schema validation
                 failure, business rule rejection, "record already exists"
  Ambiguous:     408, 409, 423 — read the specific API's docs; some mean
                 "try again", some mean "you are wrong"

  TIMEOUTS ARE THE DANGEROUS CASE. A read timeout means you do not know whether
  the work happened. Only retry if the operation is idempotent. If it is not,
  you must query for the result before deciding.

BACKOFF — always full jitter
  WRONG   sleep(2 ** attempt)
          Every client that failed at the same moment retries at the same
          moment. You have built a synchronised thundering herd.
  RIGHT   sleep(random.uniform(0, min(cap, base * 2 ** attempt)))
          AWS tested these; full jitter minimises both total work and
          completion time compared with equal-jitter and no-jitter.

  attempt 1: wait U(0, 1s)
  attempt 2: wait U(0, 2s)
  attempt 3: wait U(0, 4s)
  attempt 4: wait U(0, 8s)
  attempt 5: wait U(0, 16s)     cap at 30–60s for interactive, 300s for batch

RETRY BUDGET — the global guard
  Track (retries / total requests) over a rolling window. Above ~10%, stop
  retrying entirely and fail fast. At that point retries are amplifying an
  outage rather than riding out a blip.

HONOUR Retry-After. ALWAYS.
  Enterprise APIs — Salesforce, Graph, SAP, ServiceNow — return it and they
  mean it. Ignoring it escalates throttling and in some cases gets your
  integration user suspended.
```

Dead letter queues and the human process

A DLQ is where messages go when they cannot be processed. The technical part is easy. The part that fails in practice is that nobody owns the DLQ, nobody looks at it, and six months later it contains 40,000 messages representing real business transactions that never happened.

- [ ] Every DLQ has a named owner and an alert on depth greater than zero, or on age of the oldest message.

- [ ] Messages in the DLQ carry the failure reason, the original topic/queue, the attempt count, the timestamp and the trace ID — in headers, so you do not have to parse the body to triage.
- [ ] There is a documented process: who triages, within what SLA, what the categories are, and how to replay.
- [ ] There is a replay tool, tested, that can re-inject a selected subset after a fix — not a manual copy-paste.
- [ ] Replaying is idempotent, so re-processing a message that partially succeeded is safe.
- [ ] DLQ messages are retained long enough to be useful (14+ days) and are included in the reconciliation report.
- [ ] The DLQ itself has a DLQ policy, or at least an alert, so a failing replay does not loop forever.

PYTHON

```python
# A DLQ triage and replay tool. Ship this with the integration, not later.
import click

@click.group()
def dlq(): ...

@dlq.command()
@click.option("--topic", required=True)
@click.option("--since", default="24h")
def summary(topic, since):
    """Categorise what is in the DLQ so a human can decide quickly."""
    rows = read_dlq(topic, since)
    by_reason = Counter(r.headers.get("dlq_reason", "unknown")[:80] for r in rows)
    click.echo(f"{len(rows)} messages in {topic}.dlq since {since}\n")
    for reason, n in by_reason.most_common(20):
        click.echo(f"  {n:6d}  {reason}")
    click.echo(f"\nOldest: {min(r.timestamp for r in rows) if rows else '-'}")
    click.echo(f"Distinct keys: {len({r.key for r in rows})}")

@dlq.command()
@click.option("--topic", required=True)
@click.option("--reason-match", default=None, help="Only replay matching reasons")
@click.option("--max", "max_n", default=100)
@click.option("--dry-run/--execute", default=True)
def replay(topic, reason_match, max_n, dry_run):
    """Re-inject messages into the source topic. Default is a dry run,
    because a careless replay of 40,000 payment events is unrecoverable."""
    rows = [r for r in read_dlq(topic, "30d")
            if reason_match is None or reason_match in
               r.headers.get("dlq_reason", "")][:max_n]
    click.echo(f"{'WOULD REPLAY' if dry_run else 'REPLAYING'} {len(rows)} messages")
    if dry_run:
        for r in rows[:10]:
            click.echo(f"  {r.key}  {r.headers.get('dlq_reason','')[:60]}")
        click.echo("Re-run with --execute to proceed.")
        return
    if not click.confirm(f"Replay {len(rows)} messages to {topic}?"):
        return
    for r in rows:
        producer.produce(topic, key=r.key, value=r.value,
                         headers=list(r.headers.items()) +
                                 [("replayed_at", str(int(time.time())).encode()),
                                  ("replay_operator", os.getenv("USER","").encode())])
    producer.flush(60)
    log.warning("dlq.replayed", topic=topic, count=len(rows),
                operator=os.getenv("USER"))
```

Reconciliation — the control that catches what monitoring misses

Every integration that moves business data needs a reconciliation job. Monitoring tells you that the pipeline is running; reconciliation tells you that the data is *correct*. They are different questions and the second one is what the customer's finance team cares about.

PYTHON

```python
# Daily reconciliation. Run it, publish the result, and alert on variance.
# This single job has found more real problems than every dashboard I have built.

def reconcile(day: date) -> dict:
    src = source_system.aggregate(day)      # count + sum from the SOURCE
    tgt = target_system.aggregate(day)      # count + sum from the TARGET
    dlq_count = dlq_depth_for(day)
    rejected  = reject_file_count(day)

    report = {
        "date": day.isoformat(),
        "source_count":  src.count,
        "target_count":  tgt.count,
        "count_variance": src.count - tgt.count,
        "source_amount_minor": src.amount,
        "target_amount_minor": tgt.amount,
        "amount_variance":     src.amount - tgt.amount,
        "in_dlq":    dlq_count,
        "rejected":  rejected,
        # The accounting identity that must hold:
        "unexplained": src.count - tgt.count - dlq_count - rejected,
    }

    if report["unexplained"] != 0:
        # Records vanished without appearing anywhere. This is the alert that
        # matters; it means silent data loss.
        alert("RECONCILIATION_UNEXPLAINED_VARIANCE", severity="critical", **report)
        # Produce the actual list so a human can act, not just a count
        missing = set(src.ids) - set(tgt.ids) - set(dlq_ids(day)) - set(reject_ids(day
        write_report(f"/reports/recon/{day}/missing_ids.csv", sorted(missing))
        report["missing_sample"] = sorted(missing)[:50]
    elif report["amount_variance"] != 0:
        # Counts match but money does not: a transformation bug - rounding,
        # currency, sign, or a unit mismatch (pounds vs pence).
        alert("RECONCILIATION_AMOUNT_VARIANCE", severity="critical", **report)

    RECON_VARIANCE.labels(integration="invoice-sync").set(report["unexplained"])
    publish_report(report)
    return report
```

✓ **Reconciliation is what makes finance trust you**

Produce a daily one-page reconciliation report — records in, records out, in DLQ, rejected, unexplained — and email it to the business owner automatically from day one. Two effects. First, you detect silent data loss within 24 hours instead of at quarter-end. Second, and politically more important, the customer's finance or operations team starts trusting the integration, because they can see it balance every morning. In regulated industries this report is frequently also an audit requirement, so you get compliance value for free.

Schema evolution and contract management

TEXT

```
THE RULES THAT KEEP YOU COMPATIBLE

  BACKWARD COMPATIBLE (new consumer can read old data) — always safe
    + add an OPTIONAL field with a default
    + add a new enum value IF consumers ignore unknown values (design for this)
    + widen a type (int32 → int64, string length)
    + add a new event type on a new topic

  BREAKING — requires a new version
    - remove a field            - rename a field
    - change a type             - make an optional field required
    - change the meaning of a field while keeping its name   ⟵ the worst one,
      because nothing detects it: "amount" silently becomes gross instead of net

  VERSIONING STRATEGY THAT WORKS IN ENTERPRISES
    topic/endpoint name carries the major version:  invoice.approved.v1
    payload carries the exact schema id:            "schema":"invoice.approved.v1.3"
    v1 and v2 run IN PARALLEL during migration — publish to both, let consumers
    migrate on their own change calendar, retire v1 only when all consumers
    have confirmed. In an enterprise this takes 6–18 months. Plan for it.

  USE A SCHEMA REGISTRY if the customer has one (Confluent, Apicurio, AWS Glue).
  Set compatibility to BACKWARD or FULL and let the registry REJECT a breaking
  change at publish time rather than at 3am in a consumer you have never heard of.
```

JSON

```
{
  "type": "record",
  "name": "InvoiceApproved",
  "namespace": "com.acme.finance.v1",
  "doc": "Emitted when an invoice passes all approval steps. Idempotency key is invoic
  "fields": [
    { "name": "invoice_id",     "type": "string",
      "doc": "Source system primary key. Stable forever." },
    { "name": "source_version", "type": "long",
      "doc": "Monotonic version from the source. Consumers MUST discard older versions
    { "name": "vendor_id",      "type": "string" },
    { "name": "amount_minor",   "type": "long",
      "doc": "Amount in the smallest currency unit. NEVER a float." },
    { "name": "currency",       "type": "string", "doc": "ISO 4217" },
    { "name": "approved_at",    "type": { "type": "long", "logicalType": "timestamp-mi
    { "name": "approved_by",    "type": "string", "doc": "Hashed employee id" },
    { "name": "cost_centre",    "type": ["null", "string"], "default": null,
      "doc": "Added 2026-02 in v1.3. Optional for backward compatibility." },
    { "name": "occurred_at",    "type": { "type": "long", "logicalType": "timestamp-mi
      "doc": "When the business event happened, NOT when the message was produced." }
  ]
}
```

Money is never a float

`0.1 + 0.2 ≠ 0.3` in IEEE-754, and a cent of drift across two million invoices is a reconciliation failure that takes a week to explain to an auditor. Use integer minor units (`amount_minor: 1299` for £12.99) with a separate ISO-4217 currency field, or a fixed-point decimal type where the stack supports it. Also: never assume two decimal places — JPY has zero, KWD and BHD have three, and a hardcoded `* 100` will corrupt them. And never do arithmetic across currencies without an explicit, dated exchange rate that you record alongside the result.

Connecting to SAP

SAP runs a very large share of the world's industrial, financial and supply-chain processes, which means that if you do enterprise integration you will eventually be standing in front of an SAP system. It is a deep, old, enormously configurable platform, and the single most useful thing you can do is learn enough vocabulary to have a productive conversation with the Basis team — because they are the gatekeepers and they can tell in ninety seconds whether you know what you are asking for.

The landscape vocabulary

Term	What it is	Why you care
ECC	SAP ERP Central Component, the pre-S/4 generation (ECC 6.0 is everywhere)	Older interfaces: RFC, BAPI, IDoc. OData only if Gateway is installed and activated.
S/4HANA	The current ERP, on the HANA in-memory database	OData-first, CDS views, better APIs. On-premise or RISE-hosted cloud.
BTP	Business Technology Platform — SAP's cloud PaaS	Where SAP expects extensions to live. Includes Integration Suite, CAP, HANA Cloud, AI Core.
Basis team	The team that operates the SAP system	They create your service user, grant authorisations, open ports, activate services. Nothing happens without them.
Client (Mandant)	A logical tenant inside one SAP system, e.g. 100, 200, 800	You must specify it on every connection. Wrong client = empty results, not an error.
Transport	The mechanism for moving changes DEV → QAS → PRD	Any custom object you need must be transported, on their schedule. Weeks, not hours.
SAP GUI transaction	A screen code like SE16N, SM59, SOAMANAGER	Useful shorthand when asking for things. Knowing the codes earns credibility fast.
Authorisation object	Granular permission, e.g. S_RFC, S_TABU_DIS	"It returns no rows" is usually an authorisation problem, not a code problem.

The five ways in

1. ODATA VIA SAP GATEWAY

The modern default. SAP Gateway (embedded in S/4HANA, an add-on for ECC) exposes business objects as OData services. Ask the Basis team for the service to be activated in `/IWFND/MAINT_SERVICE` and for the metadata URL.

BASH

```
# The metadata document is your contract. Fetch it first, save it, diff it on upgrades
curl -u "$SAP_USER:$SAP_PASS" \
  --cacert /etc/ssl/acme-internal-ca.pem \
  "https://sap-gw.acme.corp:44300/sap/opu/odata/sap/API_SALES_ORDER_SRV/\$metadata" \
  -o docs/sap/API_SALES_ORDER_SRV.metadata.xml

# The catalogue of what is activated — ask for this on day one
curl -u "$SAP_USER:$SAP_PASS" \
  "https://sap-gw.acme.corp:44300/sap/opu/odata/IWFND/CATALOGSERVICE;v=2/ServiceCollec

# A real query. Note sap-client — omitting it hits the default client and
# returns data from the wrong tenant, silently.
curl -u "$SAP_USER:$SAP_PASS" \
  "https://sap-gw.acme.corp:44300/sap/opu/odata/sap/API_SALES_ORDER_SRV/A_SalesOrder\
?sap-client=100\
&\$filter=CreationDate%20ge%20datetime'2026-03-01T00:00:00'\
&\$select=SalesOrder,SoldToParty,TotalNetAmount,TransactionCurrency\
&\$top=100&\$format=json"
```

2. RFC AND BAPI

Remote Function Call is SAP's native RPC. BAPIs are the subset of function modules blessed as a stable business API — `BAPI_SALESORDER_CREATEFROMDAT2`, `BAPI_CUSTOMER_GETLIST`. You need the SAP NetWeaver RFC SDK (a licensed download from SAP, which you must get the customer to provide) and network access to the gateway ports.

PYTHON

```python
# pyrfc requires the NetWeaver RFC SDK on the machine. Build it into your image.
from pyrfc import Connection, ABAPApplicationError, CommunicationError

conn = Connection(
    ashost="sap-prd.acme.corp",   # or mshost + msserv + group for load balancing
    sysnr="00",                   # system number → gateway port 3300 + sysnr
    client="100",
    user="SVC_DOCPIPE",
    passwd=SECRET,
    lang="EN",
    # SNC (Secure Network Communications) — encryption for RFC. Production
    # SAP landscapes frequently require it and it is a separate setup task.
    snc_qop="9", snc_myname="p:CN=docpipeline, O=ACME, C=DE",
    snc_partnername="p:CN=SAPPRD, O=ACME, C=DE",
    snc_lib="/usr/local/lib/libsapcrypto.so",
)

try:
    result = conn.call("BAPI_SALESORDER_CREATEFROMDAT2",
        ORDER_HEADER_IN={"DOC_TYPE": "TA", "SALES_ORG": "1000",
                         "DISTR_CHAN": "10", "DIVISION": "00"},
        ORDER_ITEMS_IN=[{"ITM_NUMBER": "000010",
                         "MATERIAL": "000000000000012345",
                         "TARGET_QTY": "10.000"}],
        ORDER_PARTNERS=[{"PARTN_ROLE": "AG", "PARTN_NUMB": "0000001234"}],
        TESTRUN="")                       # "X" = simulate. Use it in staging.

    # 1. BAPIs return errors in RETURN, not as exceptions
    errors = [m for m in result["RETURN"] if m["TYPE"] in ("E", "A")]
    if errors:
        conn.call("BAPI_TRANSACTION_ROLLBACK")
        raise BapiError("; ".join(f"{m['ID']}{m['NUMBER']}: {m['MESSAGE']}"
                                  for m in errors))

    # 2. MOST BAPIS DO NOT COMMIT. You must call BAPI_TRANSACTION_COMMIT.
    #    Forgetting this is THE classic SAP integration bug: everything looks
    #    successful, the order number is returned, and nothing is saved.
    conn.call("BAPI_TRANSACTION_COMMIT", WAIT="X")   # WAIT='X' = synchronous
    order_id = result["SALESDOCUMENT"]
    log.info("sap.order.created", sales_order=order_id)
    return order_id

except ABAPApplicationError as e:
    log.error("sap.abap_error", key=e.key, message=e.message)
    raise
except CommunicationError as e:
    # Network, gateway, SNC. Retryable.
    log.error("sap.comm_error", message=e.message)
```

```
    raise
finally:
    conn.close()
```

3. IDOC

Intermediate Document — SAP's asynchronous, store-and-forward message format, used for decades of EDI and system-to-system flows. IDocs are typed (ORDERS05, INVOIC02, MATMAS05), versioned by SAP release, and delivered via ALE to a logical system defined in a partner profile. They are the most reliable SAP integration mechanism precisely because they are asynchronous and have built-in status tracking.

XML

```xml
<?xml version="1.0" encoding="UTF-8"?>
<ORDERS05>
  <IDOC BEGIN="1">
    <EDI_DC40 SEGMENT="1">
      <TABNAM>EDI_DC40</TABNAM>
      <DIRECT>2</DIRECT>              <!-- 1=outbound, 2=inbound -->
      <IDOCTYP>ORDERS05</IDOCTYP>     <!-- basic type -->
      <MESTYP>ORDERS</MESTYP>         <!-- message type -->
      <SNDPOR>DOCPIPE</SNDPOR>        <!-- sender port -->
      <SNDPRT>LS</SNDPRT>             <!-- LS = logical system -->
      <SNDPRN>DOCPIPELS</SNDPRN>      <!-- sender partner number -->
      <RCVPOR>SAPPRD</RCVPOR>
      <RCVPRT>LS</RCVPRT>
      <RCVPRN>PRDCLNT100</RCVPRN>     <!-- MUST match their partner profile -->
    </EDI_DC40>
    <E1EDK01 SEGMENT="1">
      <CURCY>EUR</CURCY>
      <BELNR>PO-2026-00042</BELNR>    <!-- your reference, use for idempotency -->
    </E1EDK01>
    <E1EDP01 SEGMENT="1">
      <POSEX>000010</POSEX>
      <MENGE>10.000</MENGE>
      <MENEE>PCE</MENEE>
      <E1EDP19 SEGMENT="1">
        <QUALF>002</QUALF>
        <IDTNR>000000000000012345</IDTNR>
      </E1EDP19>
    </E1EDP01>
  </IDOC>
</ORDERS05>
```

TEXT

```
IDOC STATUS CODES — memorise these six and you can triage without SAP access

  OUTBOUND                          INBOUND
  01  IDoc created                  50  IDoc added
  03  Data passed to port OK        51  Application document NOT posted  ⟵ ERROR
  12  Dispatch OK                   53  Application document posted       ⟵ SUCCESS
  26  Error during syntax check     56  IDoc with errors added
  29  Error in ALE service          63  Error passing IDoc to application
  30  IDoc ready for dispatch       64  IDoc ready to be transferred to application
  02  Error passing data to port    68  Error, no further processing

  ASK THE BASIS TEAM FOR:
    WE19 / WE02 / WE05   display and test IDocs
    BD87                 reprocess failed IDocs (your replay mechanism)
    WE20                 partner profiles — your logical system must be here
    WE21                 ports
    SM58                 failed tRFC queue — where "sent" IDocs get stuck
    SALE                 logical system definition

  THE IDEMPOTENCY STORY: put your reference in BELNR / your own field, and ask
  for a duplicate check on it. SAP will happily create two identical orders
  from two identical IDocs, and the reversal is a manual finance task.
```

4. CDS VIEWS AND DIRECT HANA ACCESS

Core Data Services views are SQL-like semantic views over SAP tables, and in S/4HANA they are the intended read model. Some are exposed as OData automatically (annotated `@OData.publish: true`); others can be read over SQL if you are granted HANA access. Direct table reads (`SE16N`-style, or JDBC to tables like `VBAK`, `BSEG`, `MARA`) are technically possible and almost always the wrong answer — you bypass all business logic, the table structures are not a stable contract, and you will be blamed for every subsequent data problem.

5. SAP INTEGRATION SUITE (CPI)

SAP's own iPaaS, on BTP. If the customer has it, they will very likely require you to route through it rather than talking to SAP directly, because it is where their integration governance, monitoring and security policies live. Learn the vocabulary: integration flow (iFlow), adapters (HTTPS, OData, SFTP, SOAP, IDoc, JMS), message mapping, content modifier, groovy script, value mapping.

The questions to ask the Basis team, verbatim

Bring this list to the first meeting. It makes you credible in ten minutes. **1.** Which system and client are we integrating with — DEV, QAS, PRD, and which client number? **2.** Is SAP Gateway installed and which OData services are activated? Can I have the catalogue service? **3.** Can you create a technical user of type SYSTEM or COMMUNICATION, not DIALOG, and what is the password policy and expiry? **4.** Which authorisation roles will it have, and can we do an authorisation trace (`ST01` / `STAUTHTRACE`) during testing? **5.** Is SNC required for RFC, and who issues the certificates? **6.** What are the gateway ports and are they open from our subnet? **7.** Is there an existing integration we can model ours on? **8.** What is the transport schedule and the change freeze calendar? **9.** What are the system's maintenance windows? **10.** For IDocs, who maintains the partner profile and can we be added?

Three SAP mistakes that cost weeks

Forgetting BAPI_TRANSACTION_COMMIT. Everything reports success, an order number comes back, and nothing is saved. Test by querying the object afterwards, always. **Using a DIALOG user.** A dialog user consumes a named-user licence, is subject to password expiry and interactive login policies, and will be locked out after failed attempts — your integration dies at 2am on day 90. Insist on a SYSTEM or COMMUNICATION user. **Reading tables directly instead of using an API.** It works in DEV, then a support pack changes a field, or a customer-specific include adds logic you bypassed, and you have been producing subtly wrong numbers for four months.

Connecting to the Big SaaS Platforms

Beyond SAP, five platforms account for most enterprise integration work. Each has a distinctive auth model, a distinctive rate limit, and one characteristic trap that catches everybody the first time. Knowing them saves you a week each.

Salesforce

Aspect	Detail
Auth	OAuth 2.0. For server-to-server use the **JWT bearer flow** with a Connected App and a certificate — no user password, no refresh token to rotate. Username-password flow still works but is being restricted and requires a security token appended to the password.
APIs	REST (per-record), SOAP (legacy), **Bulk API 2.0** (CSV, up to 150M records/day), Composite (up to 25 subrequests), Streaming/Pub-Sub API (CometD or gRPC change events), Apex REST (custom endpoints they must build)
Limits	Daily API call allowance based on licence count (e.g. 15,000 + 1,000 per licence). **This is a shared, org-wide, 24-hour rolling budget.** Governor limits apply inside Apex: 100 SOQL queries, 150 DML statements, 10s CPU per transaction.
The trap	Burning the org's daily API allowance with per-record REST calls and breaking every other integration in the company. Use Bulk API 2.0 for anything over ~2,000 records and Composite for batches. Monitor `Limits` resource and alert at 70%.

PYTHON

```python
import jwt, time, httpx

def salesforce_token(client_id: str, username: str, private_key: str,
                     login_url="https://login.salesforce.com") -> dict:
    """JWT bearer flow — no password, no refresh token, survives rotation."""
    claim = {
        "iss": client_id,          # Connected App consumer key
        "sub": username,           # the integration user
        "aud": login_url,          # MUST be test.salesforce.com for sandboxes
        "exp": int(time.time()) + 180,
    }
    assertion = jwt.encode(claim, private_key, algorithm="RS256")
    r = httpx.post(f"{login_url}/services/oauth2/token", data={
        "grant_type": "urn:ietf:params:oauth:grant-type:jwt-bearer",
        "assertion": assertion})
    r.raise_for_status()
    return r.json()        # access_token + instance_url (USE the instance_url)

# Always check your remaining budget. The header is on EVERY response.
r = client.get(f"{instance_url}/services/data/v60.0/limits")
usage = r.headers.get("Sforce-Limit-Info")      # "api-usage=12453/15000"
used, total = map(int, usage.split("=")[1].split("/"))
SF_API_USAGE.set(used / total)
if used / total > 0.8:
    log.warning("salesforce.quota", used=used, total=total,
                message="Approaching org-wide daily API limit")

# Bulk API 2.0 for volume — one "API call" for a million records
job = client.post(f"{instance_url}/services/data/v60.0/jobs/ingest", json={
    "object": "Account", "operation": "upsert",
    "externalIdFieldName": "ACME_Source_Id__c",   # the idempotency mechanism
    "lineEnding": "LF", "contentType": "CSV"}).json()
client.put(f"{instance_url}/services/data/v60.0/jobs/ingest/{job['id']}/batches",
           content=csv_bytes, headers={"Content-Type": "text/csv"})
client.patch(f"{instance_url}/services/data/v60.0/jobs/ingest/{job['id']}",
             json={"state": "UploadComplete"})
# Then poll jobs/ingest/{id} until state == JobComplete, and ALWAYS fetch
# failedResults — a job can "complete" with 40% of rows failed.
```

Microsoft 365 / Graph / SharePoint

Aspect	Detail
Auth	Entra ID (Azure AD) OAuth 2.0. App registration with **application permissions** (daemon, no user) via client credentials, using a certificate rather than a client secret in production. Requires tenant admin consent.
APIs	Microsoft Graph for almost everything (mail, files, users, Teams, calendar). SharePoint REST/CSOM for legacy on-premises. Delta queries for incremental sync. Change notifications (webhooks) with a required renewal.
Limits	Per-app, per-tenant, per-workload throttling. SharePoint/OneDrive is the strictest. Returns 429 with `Retry-After` ; sustained abuse gets you blocked for longer.
The trap	**Sites.Read.All is tenant-wide** and the customer's security team will refuse it. Use `Sites.Selected` and have them grant per-site access — but know that it must be granted per site via a separate Graph call, which surprises everyone. Also: Graph change-notification subscriptions expire (max ~3 days for most resources) and must be renewed, or your events stop silently.

PYTHON

```python
from azure.identity import CertificateCredential
import httpx

cred = CertificateCredential(
    tenant_id=TENANT_ID, client_id=CLIENT_ID,
    certificate_path="/etc/secrets/graph.pem")   # cert > secret: longer life,
                                                 # not loggable, rotatable
def graph_get_all(path: str, select: str | None = None, top: int = 999):
    token = cred.get_token("https://graph.microsoft.com/.default").token
    url = f"https://graph.microsoft.com/v1.0{path}"
    params = {"$top": top}
    if select: params["$select"] = select
    with httpx.Client(timeout=60) as c:
        while url:
            r = c.get(url, params=params,
                      headers={"Authorization": f"Bearer {token}",
                               # ConsistencyLevel is REQUIRED for $count,
                               # $search and some $filter forms on directory objects
                               "ConsistencyLevel": "eventual"})
            if r.status_code == 429:
                wait = int(r.headers.get("Retry-After", 30))
                log.warning("graph.throttled", wait_s=wait, path=path)
                time.sleep(wait)
                continue
            r.raise_for_status()
            body = r.json()
            yield from body.get("value", [])
            url, params = body.get("@odata.nextLink"), None

# DELTA QUERY — the right way to sync. First call returns everything plus a
# deltaLink; subsequent calls with that link return ONLY changes. This is the
# difference between a 6-hour full scan and a 20-second incremental.
def delta_sync(drive_id: str, state_key: str):
    link = state.get(state_key) or \
        f"https://graph.microsoft.com/v1.0/drives/{drive_id}/root/delta"
    token = cred.get_token("https://graph.microsoft.com/.default").token
    with httpx.Client(timeout=60) as c:
        while True:
            r = c.get(link, headers={"Authorization": f"Bearer {token}"})
            r.raise_for_status()
            body = r.json()
            for item in body.get("value", []):
                if "deleted" in item:
                    handle_delete(item["id"])
                else:
                    handle_upsert(item)
            if "@odata.nextLink" in body:
                link = body["@odata.nextLink"]
```

```
                continue
            state.set(state_key, body["@odata.deltaLink"])   # PERSIST THIS
            break
# If you lose the deltaLink you must do a full resync. Store it transactionally
# with the data it describes, not in memory, not in a file that gets rebuilt.
```

ServiceNow

Aspect	Detail
Auth	Basic auth (still extremely common) or OAuth 2.0. Integration user with specific roles — `rest_api_explorer` plus table-level ACLs.
APIs	Table API (generic CRUD on any table), Import Set API (staging + transform maps — the sanctioned bulk path), Scripted REST APIs (custom), Attachment API, Flow Designer with Integration Hub.
Limits	No hard published limit; governed by instance capacity and semaphores. Large unfiltered Table API queries will degrade the whole instance and their platform team will notice.
The trap	**ACLs filter rows silently.** A query returns 40 records instead of 4,000 and there is no error — the integration user simply cannot see the rest. Always validate counts against what a privileged user sees. Also: `sysparm_display_value` changes whether you get sys_ids or human labels, and switching it breaks every downstream mapping.

BASH

```
# Table API with the parameters that matter
curl -s -u "$SN_USER:$SN_PASS" \
  "https://acme.service-now.com/api/now/table/incident\
?sysparm_query=active=true^opened_at>javascript:gs.beginningOfLast7Days()^ORDERBYsys_i
&sysparm_fields=number,short_description,state,assignment_group,sys_updated_on,sys_id\
&sysparm_display_value=false\
&sysparm_exclude_reference_link=true\
&sysparm_limit=1000&sysparm_offset=0"

# sysparm_query      encoded query. ^ = AND, ^OR = OR, ^ORDERBY = sort
# sysparm_fields     ALWAYS set it. The default returns ~180 columns per row.
# sysparm_display_value  false = sys_id (stable), true = label (human, unstable)
#                        Pick one and NEVER change it. 'all' returns both.
# sysparm_limit/offset   paginate; use ORDERBYsys_id for a stable sort
# Response header X-Total-Count gives the full count when you pass
#   sysparm_count or call /api/now/stats/{table}
```

Workday

Auth is OAuth 2.0 or (still) WS-Security on SOAP. The APIs are SOAP-first (the Human Resources web service WSDL is enormous), with REST and RaaS available. **RaaS — Reports as a Service** — is the pragmatic path: a Workday analyst builds a custom report, enables web service access, and you consume it as JSON, CSV or XML over a simple URL. It is far faster than negotiating SOAP integration access, and it puts the field selection in the hands of the HR team who understand the data. The trap: Workday tenants are strictly isolated, sandbox refreshes wipe your test configuration on a published schedule, and the HR data you are touching is the most sensitive in the company — expect the longest security review of any integration you will do.

Microsoft Dynamics 365

Dataverse Web API (OData v4) plus Entra ID auth. The trap is the difference between logical names and display names, option-set integer values that differ per environment, and the `$batch` endpoint being the only sane way to do bulk operations (up to 1,000 changes per batch, in a changeset that is transactional). Plugins and workflows fire on your writes, so a bulk import can trigger thousands of background jobs and bring the environment down — always ask whether to bypass custom logic with the `MSCRM.BypassCustomPluginExecution` header, and get permission before you use it.

⚠ **The universal SaaS checklist**

For every platform, answer these before writing code: What is the auth mechanism and what expires, when? What is the rate limit, is it shared with other integrations, and how do I observe my consumption? What is the bulk path for the first full load? What is the incremental path (delta token, change stream, modified-since filter)? How do deletes propagate — do I get a deletion event, or do I have to detect absence? What is the sandbox, and how closely does it match production? What is the field-level permission model, and will it silently filter my results? Which fields are PII and what is the retention policy? Every one of these has cost someone a week.

Legacy and the Long Tail

The systems that matter most are frequently the oldest. The warehouse runs on an AS/400 application written in RPG; the general ledger posts through a nightly batch on a mainframe; the pricing team maintains a spreadsheet that six departments depend on. This is not a failure of the customer's architecture — it is thirty years of accumulated business logic that works. Your job is to integrate with it respectfully, not to propose replacing it.

SFTP, done properly

BASH

```
# The conventions that make an SFTP integration operable
/inbound/
  /incoming/        producer writes here as .tmp, then renames
  /processing/      you move the file here to claim it (atomic rename)
  /archive/2026/03/14/   processed files, retained 90 days
  /error/           files that failed validation, with a .err sidecar
/outbound/
  /ready/           files you have written and finished
  /sent/            after the partner confirms collection

# Key-based auth only. Password auth on SFTP in 2026 is an audit finding.
ssh-keygen -t ed25519 -f ./acme_sftp -C "docpipeline@vendor 2026-03"
# Send the PUBLIC key. Record the expiry date and set a calendar reminder
# 30 days before — expired SFTP keys are a top-3 cause of integration outage.

# Pin the host key. Do NOT use StrictHostKeyChecking=no; that is how you
# accept a man-in-the-middle.
ssh-keyscan -t ed25519 sftp.acme.corp >> ~/.ssh/known_hosts

# A transfer that fails loudly rather than silently
sftp -o StrictHostKeyChecking=yes \
     -o BatchMode=yes \
     -o ConnectTimeout=30 \
     -o ServerAliveInterval=15 \
     -i ./acme_sftp svc_docpipe@sftp.acme.corp <<'EOF'
cd /inbound/incoming
put invoices-20260314.csv.tmp
rename invoices-20260314.csv.tmp invoices-20260314.csv
put invoices-20260314.ctrl
bye
EOF
test $? -eq 0 || { echo "SFTP FAILED" >&2; exit 1; }
```

Alert on the file that did NOT arrive

Every monitoring system detects errors. Almost none detect absence. If the customer's nightly extract fails upstream, no file appears, no error is raised, and your pipeline reports perfect health while the business quietly runs on yesterday's data. Implement a "deadline monitor": for each expected feed, record the expected arrival window and alert when nothing has arrived by the deadline. Export it as a metric — `feed_last_received_timestamp_seconds{feed="invoices"}` — and alert on `time() - feed_last_received_timestamp_seconds > expected_interval * 1.5`. This one alert catches more real incidents than any other in a file-based integration.

Mainframe and AS/400 (IBM i)

THE GOOD PATHS

Scheduled extracts to fixed-width or CSV files over SFTP — slow, reliable, universally acceptable. IBM MQ, which most mainframe estates already run and which gives you real messaging semantics. DB2 over JDBC to a read-only replica. z/OS Connect or IBM i Integrated Web Services, which wrap existing COBOL/RPG programs as REST — ask whether it is licensed and installed, because it often is.

THE BAD PATHS

3270/5250 screen scraping. CICS transaction invocation without the owning team. Direct writes to DB2 tables bypassing the application. Anything that requires a COBOL change, because the change cycle is measured in quarters and the people who can make it are few and busy.

TEXT

```
THE MAINFRAME CONVERSATION — what to ask and what you will hear

  You: "Can we get an API?"
  Them: "There is z/OS Connect but it is not licensed / not configured."
  → Ask whether an existing CICS transaction is already exposed. Often one is.

  You: "Can we read DB2 directly?"
  Them: "You can read the replica. MIPS cost money and you will be charged."
  → Agree a query window and a cost cap. Mainframe CPU is billed per MIPS and
     a careless full-table scan has a real invoice attached. This is not a
     metaphor; someone will show you the number.

  You: "Can we get events when something changes?"
  Them: "We can write to MQ." or "We can add a row to a staging table."
  → Both are good. MQ gives you real messaging; a staging table polled every
     minute is unglamorous and works. Take either.

  DATA GOTCHAS, ALL OF WHICH YOU WILL HIT
    EBCDIC encoding (cp037, cp500) — not ASCII, not UTF-8
    Packed decimal (COMP-3) and zoned decimal with overpunch signs
    Dates as YYYYDDD (Julian), or CYYMMDD where C=0 means 19xx and 1 means 20xx
    Fixed record lengths with no delimiters at all
    Field names limited to 8-10 characters: CUSTNO, INVAMT, ORDDTE
    No NULL: empty means spaces, or zeros, or low-values, differently per field
```

Excel as an interface

It will happen. A business team maintains a spreadsheet that is genuinely the system of record for something — pricing tiers, cost centre mappings, approval thresholds — and your integration must consume it. Resist for about ten minutes, then build it properly, because the alternative is that they email it to someone who types it in.

PYTHON

```python
import openpyxl
from decimal import Decimal

def read_mapping_sheet(path: str) -> dict:
    """Consume a business-maintained workbook defensively."""
    wb = openpyxl.load_workbook(path, data_only=True, read_only=True)
    # data_only=True returns the CACHED value of formulas. If the file was
    # written by a tool rather than Excel, that cache is EMPTY and every
    # formula cell returns None. Detect it and fail with a clear message.
    if "CostCentreMap" not in wb.sheetnames:
        raise ValueError(f"Sheet 'CostCentreMap' missing. Found: {wb.sheetnames}")
    ws = wb["CostCentreMap"]

    header = [str(c.value).strip() if c.value else "" for c in next(ws.rows)]
    required = {"SourceCode", "TargetCostCentre", "ValidFrom"}
    missing = required - set(header)
    if missing:
        raise ValueError(f"Columns missing: {sorted(missing)}. Present: {header}")

    idx = {name: i for i, name in enumerate(header)}
    mapping, problems = {}, []
    for rowno, row in enumerate(ws.iter_rows(min_row=2, values_only=True), start=2):
        if all(v is None for v in row):
            continue                                  # blank spacer row
        src = row[idx["SourceCode"]]
        if src is None:
            continue
        src = str(src).strip()
        # Excel helpfully turns "00123" into the number 123. Restore it.
        if src.isdigit() and len(src) < 5:
            src = src.zfill(5)
        tgt = row[idx["TargetCostCentre"]]
        if tgt is None:
            problems.append(f"row {rowno}: SourceCode {src} has no target")
            continue
        if src in mapping:
            problems.append(f"row {rowno}: duplicate SourceCode {src}")
        mapping[src] = str(tgt).strip()

    if problems:
        # Email the business owner the EXACT rows to fix. Do not silently drop.
        notify_owner("CostCentreMap validation", problems)
    if len(problems) > len(mapping) * 0.1:
        raise ValueError(f"{len(problems)} problems in {len(mapping)} rows — "
                         f"refusing to load. See the emailed report.")
    return mapping

# AND: version the file. Copy every accepted version to immutable storage with
```

```
# a timestamp and a hash, so that when someone asks "why did cost centre 4411
# change in March" you can answer it.
```

> **Turn the spreadsheet into a contract**
> Give the business team a template with a locked header row, data validation on the key columns, and a version cell they must increment. Validate on upload and reply with a precise error list within a minute. You have now converted an uncontrolled file into a governed interface without taking anything away from the people who own the data — which is the only version of this that they will accept.

Screen scraping and RPA, as last resorts

Sometimes there genuinely is no interface: a vendor portal with no API, a terminal application, a mainframe green screen. UiPath, Power Automate Desktop, Playwright or a 5250 emulator can automate it. Be honest about what you are buying: this is the most fragile integration that exists. It breaks when a button moves, when a session times out differently, when a CAPTCHA appears, when the vendor redesigns. Treat it as a bridge with an explicit expiry date, instrument it heavily (screenshot on failure, step-level timing, success rate), run it on a schedule with a human-visible failure path, and keep pressing for a real interface. Also check the terms of service — automating a vendor's portal is sometimes a contractual breach, and that is a conversation for the customer's legal team, not for you to decide alone.

Databases as Integration Points

"Just give us read access to the database" is the request that sounds reasonable and is usually a trap. Sometimes it is the right answer. Knowing which is which, and knowing the modern alternative — log-based change data capture — separates an integration engineer from someone who writes SELECT statements.

Why direct database access is usually wrong

YOU BYPASS THE BUSINESS LOGIC

An application's tables are its internal state, not its API. The status column means something only in combination with three other tables and a rule implemented in code. Reading it directly means reimplementing that logic and getting it subtly wrong.

THE SCHEMA IS NOT A CONTRACT

The owning team will rename a column in a routine release, because from their perspective it is private. They are right. Your integration breaks and the postmortem concludes it was your fault for depending on it.

YOU CREATE LOAD YOU DO NOT CONTROL

A badly written join against a production OLTP database at 09:00 on a Monday is an outage with your name on it. DBAs know this, which is why they resist.

POLLING DOES NOT SCALE AND MISSES DELETES

`WHERE updated_at > :last` requires a reliable updated_at (often missing or set by triggers that do not fire on bulk updates), misses hard deletes entirely, and either polls too often (load) or too rarely (latency).

When it is acceptable

A read-only replica, with a dedicated login, a statement timeout, a resource governor or workload group, agreed query patterns, and written acknowledgement that the schema may change. Or a purpose-built view or materialised view that the owning team maintains as a deliberate, versioned interface — which is really an API that happens to speak SQL, and that is fine.

Change data capture with Debezium

CDC reads the database's own transaction log — PostgreSQL WAL, MySQL binlog, SQL Server CDC tables, Oracle LogMiner or XStream — and emits a stream of row-level change events. It captures inserts, updates *and deletes*, in commit order, with before and after images, at very low cost to the source database because reading the log is what replicas do anyway.

JSON

```json
{
  "name": "acme-erp-invoices",
  "config": {
    "connector.class": "io.debezium.connector.postgresql.PostgresConnector",
    "database.hostname": "erp-replica.acme.corp",
    "database.port": "5432",
    "database.user": "debezium",
    "database.password": "${file:/secrets/db.properties:password}",
    "database.dbname": "erp",
    "topic.prefix": "acme.erp",

    "plugin.name": "pgoutput",
    "slot.name": "debezium_docpipeline",
    "publication.name": "dbz_docpipeline",
    "publication.autocreate.mode": "filtered",

    "table.include.list": "public.invoices,public.invoice_lines,public.vendors",
    "column.exclude.list": "public.vendors.bank_account,public.vendors.tax_id",

    "snapshot.mode": "initial",
    "snapshot.fetch.size": "10240",
    "incremental.snapshot.chunk.size": "4096",

    "heartbeat.interval.ms": "10000",
    "heartbeat.action.query": "INSERT INTO dbz_heartbeat(ts) VALUES (now()) ON CONFLIC

    "transforms": "unwrap,route",
    "transforms.unwrap.type": "io.debezium.transforms.ExtractNewRecordState",
    "transforms.unwrap.drop.tombstones": "false",
    "transforms.unwrap.delete.handling.mode": "rewrite",
    "transforms.unwrap.add.fields": "op,source.ts_ms,source.lsn,source.txId",
    "transforms.route.type": "org.apache.kafka.connect.transforms.RegexRouter",
    "transforms.route.regex": "acme\\.erp\\.public\\.(.*)",
    "transforms.route.replacement": "acme.erp.$1.v1",

    "key.converter": "io.confluent.connect.avro.AvroConverter",
    "value.converter": "io.confluent.connect.avro.AvroConverter",
    "value.converter.schema.registry.url": "http://schema-registry:8081"
  }
}
```

JSON

```
// What a Debezium change event looks like. 'op' is the whole story:
// c=create  u=update  d=delete  r=read (initial snapshot)  t=truncate
{
  "before": { "id": 4411, "status": "approved", "amount_minor": 125000 },
  "after":  { "id": 4411, "status": "paid",     "amount_minor": 125000 },
  "source": {
    "version": "2.6.0.Final", "connector": "postgresql",
    "name": "acme.erp", "ts_ms": 1773483667411,
    "snapshot": "false", "db": "erp", "schema": "public", "table": "invoices",
    "txId": 8821344, "lsn": 42949673216
  },
  "op": "u",
  "ts_ms": 1773483667498
}
```

The replication slot is a loaded gun

In PostgreSQL, a logical replication slot makes the database retain WAL segments until the consumer confirms it has read them. If your Debezium connector stops — a crash, a Kafka outage, an expired credential, a long weekend — WAL accumulates and the source database's disk fills. When it fills, the customer's production ERP stops accepting writes. This is a genuine, documented way to take down a system you were only supposed to read from. Mitigations, all of them, not one: monitor `pg_replication_slots.confirmed_flush_lsn` lag and alert hard at a few gigabytes; set `max_slot_wal_keep_size` (PostgreSQL 13+) so the database drops the slot rather than dying; configure a heartbeat query so low-traffic tables still advance the slot; and have a documented runbook for dropping the slot and re-snapshotting. Tell the DBA about all of this before you create the slot, in writing. They will respect you for it.

CDC versus polling versus triggers

	Log-based CDC	Polling on updated_at	Trigger to outbox table
Captures deletes	Yes	No	Yes
Captures every intermediate state	Yes	No (misses rapid changes)	Yes
Load on source	Very low	Proportional to poll frequency	Write amplification on every change
Latency	Sub-second	Poll interval	Sub-second to the table; then polled
Requires schema change	No	Needs a reliable updated_at	Yes — a table and triggers
Requires DBA cooperation	High (replication privileges, slot)	Low	Medium (DDL + triggers)
Operational risk	Slot/WAL growth	Query load, missed rows	Trigger bugs affect production writes
Works on	Postgres, MySQL, SQL Server, Oracle, Db2, Mongo	Anything	Anything with triggers

⚠ **CDC gives you rows, not events**

A change event says "the status column changed from approved to paid". It does not say "an invoice was paid", which is what consumers actually need — and reconstructing the business meaning from row deltas downstream puts your domain logic in the wrong place and makes you permanently coupled to the source schema. Where you can influence the design, prefer the outbox pattern: the application writes an explicit, well-named business event into an outbox table inside the same transaction, and CDC streams that table. You get CDC's reliability with a stable, meaningful contract. Where you cannot influence the design, do the row-to-event translation in exactly one place, immediately after ingestion, and never let raw CDC topics reach business consumers.

iPaaS and Middleware

Most large enterprises already own an integration platform, and a team whose job is to govern it. You will be told to use it. Your instinct will be to write a small clean service instead. Sometimes that instinct is right; more often, going through their platform is the faster path to production, because it inherits approvals, monitoring, credentials management and operational ownership that would otherwise take you months to establish.

The landscape

Platform	Shape	Strength	Where it hurts
MuleSoft	Anypoint Studio, DataWeave, API-led connectivity	Powerful transformation language, strong API management, big connector library	Expensive; DataWeave is a language to learn; heavyweight for simple flows
Dell Boomi	Browser-based visual flows, Atoms (runtimes)	Fast to build, low barrier, good SaaS connectors	Version control and code review are awkward; complex logic becomes unreadable
Azure Logic Apps / Integration Services	Visual workflows, 1000+ connectors, consumption or standard plans	Native Azure identity and networking; cheap for low volume; good for event-driven glue	Debugging a large designer workflow is painful; per-action billing surprises
SAP Integration Suite (CPI)	iFlows, adapters, message mapping, Groovy scripts	Pre-built SAP content; the sanctioned path in SAP shops	SAP-centric; slow iteration; Groovy is the escape hatch for everything hard
Workato / Tray / Zapier	Low-code recipes	Business users can build; extremely fast for simple SaaS-to-SaaS	Governance, data residency, and "who owns this recipe" become real problems
Apache Camel	Open-source library/framework (Java), 300+ components	EIP patterns as a DSL, runs anywhere, no licence, testable as code	You own the runtime, the monitoring and the on-call
IBM App Connect / ACE	Message flows, ESQL	Deep MQ and mainframe integration	Old, scarce skills, long change cycles

TEXT

```
// Apache Camel makes the EIP patterns literal. This is one file that a
// reviewer can read, a test can exercise, and git can diff — which is the
// argument for it over a visual tool.
public class InvoiceRoute extends RouteBuilder {
  @Override
  public void configure() {

    // Global error handling with backoff and a dead letter channel
    errorHandler(deadLetterChannel("jms:queue:invoice.dlq")
        .maximumRedeliveries(5)
        .redeliveryDelay(1000)
        .backOffMultiplier(2)
        .maximumRedeliveryDelay(60000)
        .useExponentialBackOff()
        .retryAttemptedLogLevel(LoggingLevel.WARN)
        .onPrepareFailure(exchange -> {
            exchange.getIn().setHeader("dlq_reason",
                exchange.getProperty(Exchange.EXCEPTION_CAUGHT, Exception.class)
                        .getMessage());
            exchange.getIn().setHeader("dlq_at", System.currentTimeMillis());
        }));

    // Non-retryable failures must NOT be retried five times
    onException(ValidationException.class)
        .handled(true)
        .log(LoggingLevel.ERROR, "Validation failed: ${exception.message}")
        .to("jms:queue:invoice.rejected");

    from("sftp://svc@sftp.acme.corp/inbound/incoming"
         + "?privateKeyFile=/secrets/sftp_key"
         + "&include=invoices-.*\\.csv"
         + "&move=../archive/${date:now:yyyy/MM/dd}"
         + "&moveFailed=../error"
         + "&readLock=changed&readLockMinAge=10s"    // partial-file guard
         + "&delay=60000")
      .routeId("invoice-file-ingest")
      .log("Picked up ${header.CamelFileName}")

      .unmarshal().csv()                                    // Translator
      .split(body()).streaming()                            // Splitter
        .filter(simple("${body[status]} != 'TEST'"))        // Filter
        .bean(InvoiceValidator.class, "validate")
        .enrich("direct:lookupVendor",                      // Content enricher
                new VendorMergeStrategy())
        .choice()                                           // Router
          .when(simple("${body.amountMinor} > 1000000"))
            .to("jms:queue:invoice.approval.required")
          .otherwise()
```

```
            .idempotentConsumer(simple("${body.invoiceId}"), // Idempotent receiver
                JdbcMessageIdRepository.jpaMessageIdRepository(ds, "invoice-ids"))
            .to("jms:queue:invoice.post")
        .end()
      .end()
      .log("Completed ${header.CamelFileName}: "
        + "${header.CamelSplitSize} rows");
  }
}
```

Build versus buy versus "live with theirs"

1. **If the customer has a platform and a team, use it for flows that fit it.**
   Simple mapping, routing and scheduled transfers belong there. You inherit their monitoring, their credential vault, their change process and — crucially — their operational ownership after you leave.

2. **Write code for anything with real logic.**
   Complex business rules, anything iterative, anything that needs proper testing, anything performance-sensitive. Expressing a twelve-branch algorithm in a visual designer produces something nobody can review or maintain.

3. **Never split one flow across both.**
   Half in Boomi and half in your service is the worst outcome: two debugging surfaces, two deployment processes, two teams, and an error-handling seam where messages vanish. Pick the boundary at a clean interface — their platform calls your HTTP endpoint, or publishes to a queue you consume.

4. **Expose a clean interface either way.**
   Whether you build or they build, the contract between the two sides is a documented API or message schema with a version, not a shared database table or an undocumented file format.

5. **Count the cost of their platform honestly.**
   Per-connection, per-message or per-flow licensing is real money, and some platforms charge per endpoint. If your design creates 40 new connections, find out what that costs before you present it.

How to win the "use our ESB" conversation

Do not argue about technology. Ask three questions: what is the lead time for a change in the platform, who operates it after go-live, and what happens when a message fails at 2am? If the answers are "six weeks", "the integration CoE" and "they triage it next business day", and your use case needs faster iteration or real-time error handling, you have a factual basis for an exception — and you can offer a compromise: your service handles the real-time path, their platform handles the batch and governance path, with a documented interface between them. That proposal gets approved far more often than "we would prefer to write our own".

Security and Governance of Integrations

Integrations move data across trust boundaries, which makes them the most security-sensitive code you will write. They are also where security reviews stall projects for weeks, so understanding what the reviewer is looking for — and pre-empting it — is directly valuable to your delivery timeline.

Identity: what to ask for and why

Mechanism	Use when	Rotation burden	Reviewer's concern
OAuth 2.0 client credentials + certificate	Server-to-server, modern platforms	Certificate lifetime (1–2 years)	Low. This is the preferred answer.
OAuth 2.0 client credentials + secret	Server-to-server where certs are not supported	Secret expiry, often 6–24 months	Secret storage and rotation process
JWT bearer / private key JWT	Salesforce, some banks, SAP BTP	Key rotation	Key storage; usually well received
mTLS	Bank and government partners, internal zero-trust	Certificate lifetime	PKI ownership, revocation, who issues
API key	Simple SaaS, internal services	Manual	Transmitted how? Scoped how? Rotatable?
Basic auth (service account)	Legacy, ServiceNow, SAP, SFTP-adjacent	Password policy, often 90 days	Significant. Expect pushback; propose exemption with compensating controls.
SSH key	SFTP	Manual, often never	Key length, algorithm, expiry, who holds the private key
Delegated user credentials	Never, for an integration	N/A	Immediate rejection. The integration breaks when that person leaves.

Never integrate as a named human

It happens constantly: "just use my account for now". The consequences are predictable. The person changes their password and the integration dies. They leave the company and it dies permanently at 3am. Their MFA policy changes and it dies. Every action in the audit log is attributed to them, so an incident investigation implicates an innocent person. And the integration inherits their full permissions, which is wildly more than it needs. Insist on a dedicated service identity with least-privilege scopes, named clearly (`svc_docpipeline_prd`), owned by a team rather than a person, documented in the customer's identity register, and exempted from interactive-login policies.

Secrets

- [ ] Secrets live in the customer's vault — Azure Key Vault, HashiCorp Vault, AWS Secrets Manager, CyberArk — not in environment variables baked into an image, not in a config file, never in git.
- [ ] Prefer workload identity (managed identity, IRSA, workload identity federation) so that there is no secret at all. This is the answer that makes security reviewers happy and it is increasingly available.
- [ ] Every credential has a recorded expiry date and an owner, and there is a calendar reminder 30 days before. Expired credentials are a top-three cause of integration outages, and they always expire on a holiday.
- [ ] Rotation must be possible without downtime: support two valid credentials simultaneously during the overlap, or re-read the secret on a schedule rather than only at startup.
- [ ] Expose credential expiry as a metric: `integration_credential_expiry_timestamp_seconds{system="sap"}` , and alert at 30, 14 and 7 days.
- [ ] Secrets never appear in logs, traces, error messages, support bundles or exception payloads. Test this explicitly by logging an object that contains one.
- [ ] Scope every credential to the minimum: specific objects, specific operations, read-only where possible, IP-restricted where supported.

PYTHON

```python
# Credential expiry as a first-class signal. Three lines of instrumentation
# that prevent the most boring and most common outage in enterprise integration.
from prometheus_client import Gauge

CRED_EXPIRY = Gauge("integration_credential_expiry_timestamp_seconds",
                    "Unix timestamp when this credential expires",
                    ["system", "credential", "environment"])

def refresh_credential_expiry_metrics():
    for cred in credential_registry.all():
        CRED_EXPIRY.labels(cred.system, cred.name, cred.env).set(cred.expires_at)

# Alert rule:
#   (integration_credential_expiry_timestamp_seconds - time()) / 86400 < 30
# Annotation: "Credential {{$labels.credential}} for {{$labels.system}} expires
#              in {{$value | humanizeDuration}}. Owner: see the credential
#              register. Renewal lead time with the customer is 10 working days."
#
# That last sentence is the one that matters — knowing the LEAD TIME is why
# you alert at 30 days and not at 3.
```

Network controls you will negotiate

IP ALLOW-LISTING

They will want a fixed set of source IPs. In cloud, that means a NAT gateway with static IPs or a dedicated egress. Agree the list, document it, and commit to notice before changes — then actually give notice, because a surprise IP change is an outage.

FORWARD PROXY

All outbound through `proxy.corp:8080`, often with authentication and TLS inspection. Your HTTP client, your SDK, your container runtime and your language runtime all need to be told, and they all use different variables. Test the full chain.

MTLS

Both sides present certificates. Needs a PKI decision: their internal CA, a public CA, or self-signed with pinning. Ask who issues, what the lifetime is, and what the renewal process is — then put the expiry in your metrics.

PRIVATE CONNECTIVITY

Private Link, VPC peering, ExpressRoute, VPN. Slower to set up (weeks) but often the only acceptable option for data-sensitive flows. Get the CIDR ranges early to avoid overlap.

TLS INSPECTION

A middlebox terminates and re-signs TLS. Your code sees their CA, not the real one. Certificate pinning breaks. The fix is to trust their CA explicitly — never to disable verification.

DLP AND EGRESS FILTERING

Data loss prevention agents may block payloads containing patterns that look like card numbers or national IDs — including your legitimate business data. Find out early; the error is usually an opaque connection reset.

BASH

```
# Proxy configuration is never just one variable. All of these matter.
export HTTP_PROXY=http://proxy.acme.corp:8080
export HTTPS_PROXY=http://proxy.acme.corp:8080
export NO_PROXY=localhost,127.0.0.1,.acme.corp,.svc.cluster.local,10.0.0.0/8
export http_proxy=$HTTP_PROXY          # lowercase too; libraries differ
export https_proxy=$HTTPS_PROXY
export no_proxy=$NO_PROXY

# Trusting an internal CA — the correct fix for TLS inspection
export SSL_CERT_FILE=/etc/ssl/certs/ca-certificates.crt   # OpenSSL, Go
export REQUESTS_CA_BUNDLE=/etc/ssl/certs/ca-certificates.crt  # Python requests
export NODE_EXTRA_CA_CERTS=/etc/ssl/acme-internal-ca.pem      # Node
# Java: keytool -importcert -alias acme-ca -keystore $JAVA_HOME/lib/security/cacerts
# .NET: the OS trust store on Linux is /etc/ssl/certs

# In the image, merge their CA into the system bundle at build time:
#   COPY acme-internal-ca.pem /usr/local/share/ca-certificates/acme.crt
#   RUN update-ca-certificates

# NEVER:  verify=False / NODE_TLS_REJECT_UNAUTHORIZED=0 / -k / --insecure
# It will be found in a security review, it will be a finding, and you will
# have to redo the work anyway — after having destroyed your credibility.
```

Data governance

- [ ] Classify every field you move: public, internal, confidential, restricted/PII/PHI. Put it in a table in the integration specification. Reviewers approve documents, not intentions.
- [ ] Minimise: transfer only the fields the target genuinely needs. "We take the whole record because it is easier" is both a security finding and an unnecessary coupling.
- [ ] Know the lawful basis and the residency rules. Moving EU employee data to a US-hosted service is a DPA question, not an engineering decision, and it must be raised in week one or it blocks go-live.
- [ ] Audit trail: who accessed what, when, from where — including your integration's own reads. Many regulated customers require this and it is far easier to build in than to retrofit.
- [ ] Retention and erasure: if a data subject requests erasure, can the customer find and delete their records in every store your integration created, including queues, DLQs, archives and logs? Design for this; it will be asked.
- [ ] Pseudonymise where you can: a salted hash of an employee ID still supports joins and deduplication without carrying an identifier.

- [ ] Encrypt in transit (TLS 1.2+ everywhere, no exceptions) and at rest (the platform's KMS, with the customer's keys where they require it).

Rate Limits, Quotas and Designing Within a Budget

Every API you integrate with has a budget, and in an enterprise that budget is usually shared with integrations built by people you will never meet. Exceeding it does not just slow you down — it breaks other teams' systems, which is a much worse conversation than a performance problem.

Limit models and how to detect which one you face

Model	Behaviour	Example	Design response
Fixed window	N calls per calendar hour/day, resets abruptly	Salesforce 24h org limit	Track consumption; reserve headroom for other teams; schedule bulk work after reset
Sliding window	N calls in any trailing period	Many REST APIs	Client-side rate limiter matching their window
Token bucket	Steady refill with burst capacity	AWS, Stripe, many gateways	Burst for the initial load, then settle to the refill rate
Concurrency limit	Max N simultaneous requests	SAP, databases, ServiceNow semaphores	Bounded worker pool, not rate limiting
Cost/complexity based	Points per query by expense	GraphQL, Shopify, Microsoft Graph some endpoints	Request fewer fields; measure cost per query type
Undocumented / adaptive	Throttles based on load, no published number	Internal enterprise systems, SharePoint	Adaptive concurrency: back off on 429, cautiously increase on success

PYTHON

```python
# A client-side limiter plus adaptive concurrency. Use BOTH: the limiter
# enforces the published rate, the adaptive controller discovers the real one.
import asyncio, time, random

class TokenBucket:
    def __init__(self, rate_per_sec: float, burst: int):
        self.rate, self.capacity = rate_per_sec, burst
        self.tokens = float(burst)
        self.updated = time.monotonic()
        self._lock = asyncio.Lock()

    async def acquire(self, n: int = 1):
        while True:
            async with self._lock:
                now = time.monotonic()
                self.tokens = min(self.capacity,
                                  self.tokens + (now - self.updated) * self.rate)
                self.updated = now
                if self.tokens >= n:
                    self.tokens -= n
                    return
                deficit = (n - self.tokens) / self.rate
            await asyncio.sleep(deficit + random.uniform(0, 0.05))

class AdaptiveConcurrency:
    """AIMD - additive increase, multiplicative decrease. The same algorithm
    TCP uses for congestion control, applied to an undocumented API limit."""
    def __init__(self, start=4, minimum=1, maximum=64):
        self.limit = start
        self.min, self.max = minimum, maximum
        self._sem = asyncio.Semaphore(start)
        self._since_change = 0

    def on_success(self):
        self._since_change += 1
        if self._since_change >= 20 and self.limit < self.max:
            self.limit += 1                      # additive increase
            self._sem.release()
            self._since_change = 0
            CONCURRENCY_LIMIT.set(self.limit)

    def on_throttle(self):
        new = max(self.min, self.limit // 2)     # multiplicative decrease
        for _ in range(self.limit - new):
            asyncio.create_task(self._sem.acquire())
        self.limit = new
        self._since_change = 0
        CONCURRENCY_LIMIT.set(self.limit)
```

```
        log.warning("concurrency.reduced", new_limit=new)

bucket = TokenBucket(rate_per_sec=8, burst=20)
concurrency = AdaptiveConcurrency()

async def call(url):
    await bucket.acquire()
    async with concurrency._sem:
        r = await client.get(url)
        # Read the limit headers — most APIs tell you exactly where you stand
        remaining = r.headers.get("X-RateLimit-Remaining") \
                 or r.headers.get("RateLimit-Remaining")
        if remaining is not None:
            API_QUOTA_REMAINING.labels(api="crm").set(int(remaining))
        if r.status_code == 429:
            concurrency.on_throttle()
            wait = float(r.headers.get("Retry-After", 30))
            THROTTLED.labels(api="crm").inc()
            await asyncio.sleep(wait + random.uniform(0, 2))
            raise Throttled(wait)
        concurrency.on_success()
        return r
```

Designing within the budget

1. **Measure the budget first.**
   Call the limits endpoint, read the headers, ask the platform owner. Write the number into the design document. "Salesforce org limit 45,000/day, current baseline consumption from other integrations 28,000/day, our budget 8,000/day with 20% headroom."

2. **Use the bulk path for bulk work.**
   One Bulk API job for a million records versus a million REST calls is the difference between 1 API call and 1,000,000. This single decision usually determines whether the project is feasible.

3. **Go incremental as fast as possible.**
   Delta queries, modified-since filters, change streams. A full scan should happen once, at go-live, and then never again.

4. **Cache reference data aggressively.**
   Country lists, product masters, cost centre mappings, user directories. These change daily at most. Caching them for an hour removes most of your call volume.

5. **Batch and coalesce.**

Composite requests, `$batch`, multi-record payloads, and debouncing — if the same record changes five times in a minute, sync it once.

6. **Schedule around their peak.**
Run bulk work in their quiet window, and ask what that is. Then check that their quiet window is not also when their backup, their batch and three other integrations run.

7. **Degrade, do not die.**
When you hit the limit, queue the work and slow down. Do not fail the user's request if the data can be eventually consistent.

8. **Instrument consumption as a percentage of budget**
and alert at 70%, before you are the reason someone else's integration fails.

✓ **Be a good neighbour, visibly**
Tell the platform owner your expected call volume before you deploy, show them the dashboard of your actual consumption after you deploy, and reduce it when you can. In every enterprise, API quota is a shared commons, and the team that treats it carelessly becomes the team that gets throttled, audited and eventually removed. Two emails and one dashboard panel buy you an enormous amount of goodwill with the people who control whether your integration keeps running after you leave.

Testing, Cutover and Operations

Integration testing is harder than unit testing because the interesting behaviour lives in systems you do not control, cannot easily reset, and must not break. Cutover is harder still, because it happens once, under time pressure, in a window that was negotiated three months ago and cannot slip. Both are solvable with preparation, and preparation is the entire skill.

The testing pyramid for integrations

TEXT

```
LEVEL 5  PRODUCTION VERIFICATION          few, continuous
         Synthetic transactions through the real path, end to end, every
         15 minutes. Reconciliation daily. This is the only test that proves
         the integration works TODAY.

LEVEL 4  END TO END IN THE CUSTOMER'S QAS      few, slow, scheduled
         Real systems, real-ish data, real network path. Expensive to run,
         slow to reset, shared with other projects. Book time in advance.

LEVEL 3  INTEGRATION AGAINST SANDBOXES         some, nightly
         Salesforce sandbox, SAP QAS, a vendor's test tenant. Tests auth,
         payload shape and real error behaviour. Flaky: sandboxes get
         refreshed, throttled and taken down without notice.

LEVEL 2  CONTRACT TESTS                        many, every commit
         Validate your requests against their OpenAPI/WSDL/Avro schema and
         their recorded responses against your parser. Catches drift without
         needing the real system available.

LEVEL 1  UNIT TESTS WITH RECORDED FIXTURES     hundreds, milliseconds
         Real captured payloads — including the ugly ones — replayed against
         your mapping and validation logic. This is where you encode every
         production surprise you ever encounter.
```

PYTHON

```python
# Level 1: fixtures captured from reality, including every failure you have met.
# Every production incident should add a file to this directory. That is how
# the suite becomes genuinely protective rather than decorative.
#
#   tests/fixtures/sap/
#     salesorder_create_success.xml
#     salesorder_create_bapi_error_missing_material.xml
#     salesorder_create_200_but_return_type_E.xml     ← the 200-is-not-success case
#     salesorder_create_html_login_page.html          ← session expired, got HTML
#     salesorder_create_truncated.xml                 ← connection dropped mid-body
#     odata_v2_date_format.json
#     odata_empty_results.json
#     odata_error_csrf_required.json

import pytest, pathlib
FIX = pathlib.Path(__file__).parent / "fixtures" / "sap"

@pytest.mark.parametrize("fixture,expected", [
    ("salesorder_create_success.xml",                       "ok"),
    ("salesorder_create_bapi_error_missing_material.xml",   "BapiError"),
    ("salesorder_create_200_but_return_type_E.xml",         "BapiError"),
    ("salesorder_create_html_login_page.html",              "AuthExpired"),
    ("salesorder_create_truncated.xml",                     "ParseError"),
])
def test_response_handling(fixture, expected):
    raw = (FIX / fixture).read_bytes()
    if expected == "ok":
        assert parse_salesorder_response(raw).order_id
    else:
        with pytest.raises(EXCEPTIONS[expected]):
            parse_salesorder_response(raw)

# Level 2: contract test against THEIR schema, not your assumption of it
def test_request_validates_against_wsdl():
    schema = load_wsdl_schema("docs/sap/salesorder.wsdl")
    request = build_salesorder_request(sample_order())
    schema.validate(request)          # fails loudly if their schema changed

# Level 3: the real sandbox, marked so CI can skip it when it is down
@pytest.mark.integration
@pytest.mark.skipif(not os.getenv("SAP_QAS_HOST"), reason="no sandbox configured")
def test_create_order_in_qas():
    order_id = create_sales_order(sample_order(), testrun=True)   # TESTRUN='X'
    assert order_id
```

✦ **Record real traffic, replay it forever**

The first time you talk to a customer's system, capture everything — request, response, headers, status, timing — into a fixture directory, with secrets redacted. Those recordings become your test suite and your documentation, and they let you develop on a train with no VPN. Tools like VCR.py, `responses`, WireMock or a simple logging proxy make this almost free. The customer's sandbox will be down for two days during your sprint; the recordings will not be.

Mock services for development

YAML

```
# WireMock stub: stand up a fake SAP Gateway so twelve developers are not
# fighting over one sandbox. Model the ERROR cases, not just the happy path.
mappings:
  - request:
      method: GET
      urlPathPattern: "/sap/opu/odata/sap/API_SALES_ORDER_SRV/A_SalesOrder"
      queryParameters:
        sap-client: { equalTo: "100" }
    response:
      status: 200
      headers: { Content-Type: application/json }
      bodyFileName: odata_salesorder_page1.json
      fixedDelayMilliseconds: 180        # model their real latency

  # Throttling — your retry logic must be exercised in CI, not discovered in prod
  - request:
      method: GET
      urlPathPattern: "/sap/opu/odata/.*"
      headers: { X-Test-Scenario: { equalTo: "throttle" } }
    response:
      status: 429
      headers: { Retry-After: "12" }

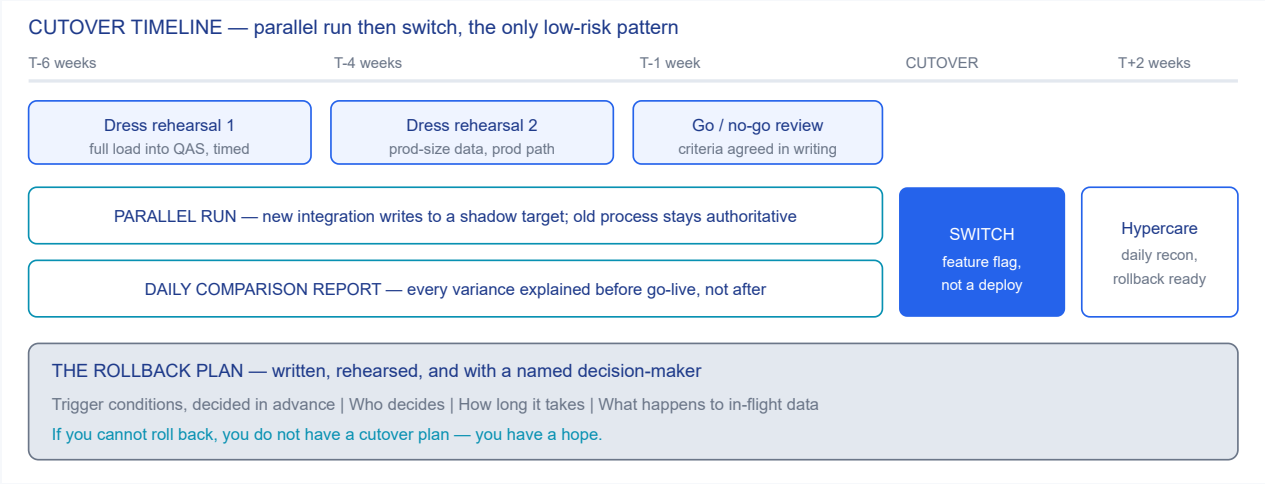
  # CSRF expiry — the SAP-specific dance
  - request:
      method: POST
      urlPathPattern: "/sap/opu/odata/.*"
      headers: { x-csrf-token: { equalTo: "stale" } }
    response:
      status: 403
      headers: { x-csrf-token: "Required" }

  # Slow response — proves your timeout is actually configured
  - request:
      method: GET
      urlPathPattern: "/sap/opu/odata/.*"
      headers: { X-Test-Scenario: { equalTo: "slow" } }
    response:
      status: 200
      fixedDelayMilliseconds: 45000

  # Session expiry returning an HTML login page instead of JSON.
  # This WILL happen in production and it breaks naive JSON parsers.
  - request:
      method: GET
      urlPathPattern: "/sap/opu/odata/.*"
      headers: { X-Test-Scenario: { equalTo: "session-expired" } }
    response:
```

```
status: 200
headers: { Content-Type: "text/html" }
body: "<html><body><form action='/sap/bc/sec/login'> … </form></body></html>"
```

Data migration and cutover



Parallel run with a flag-based switch. The comparison report is what converts opinion into a go/no-go decision.

1. **Profile the source data before you design the migration.**
   Row counts, distinct values, null rates, duplicates, orphans, encoding anomalies, dates in the future, negative amounts. You will find that 4% of records violate a constraint everyone assured you was enforced. Find it in week two, not in the cutover window.

2. **Define the migration as idempotent and resumable.**
   It will fail halfway at least once. Keyed upserts, a checkpoint table, and the ability to restart from the last committed batch.

3. **Rehearse at full scale, twice.**
   A migration that takes 90 minutes in test with 10% of the data takes fifteen hours with all of it, and your window is four. Measure, then optimise, then measure again. The second rehearsal is where you discover the problems the first rehearsal's fixes introduced.

4. **Run in parallel.**
   New integration writes to a shadow target while the old process remains authoritative. Compare outputs daily. Every variance must be explained — not waved away — before go-live.

5. **Agree go/no-go criteria in writing, in advance.**
   "Variance under 0.01%, zero unexplained records, full load completes in under 3 hours, all reconciliation checks green." Decided when everyone is calm, not at 02:00 during the window.

6 **Switch with a feature flag, not a deployment.**
A flag flips in seconds and flips back in seconds. A deployment takes twenty minutes and a rollback takes longer under pressure.

7 **Freeze changes around the window.**
No other releases, no infrastructure changes, no certificate rotations. Get this agreed with the customer's change board as part of the window booking.

8 **Hypercare.**
Two weeks of daily reconciliation, a daily stand-up with the business owner, and the rollback still available. This is also when you train their support team, because the questions are real.

TEXT

```
CUTOVER RUNBOOK — the format that works. Every line has an owner and a
duration, and the whole thing is rehearsed before the real thing.

  CUTOVER: Invoice integration, ACME, Sat 14 Mar 2026, window 22:00–04:00 CET
  Decision maker: J. Weber (ACME Finance Ops)    Rollback decision by: 03:00

  T-24h  [ ] Confirm change freeze in effect                  P.Shah    5m
  T-24h  [ ] Confirm SAP QAS refresh will NOT run this weekend  Basis     -
  T-2h   [ ] Final reconciliation of parallel run              T.Okafor  30m
  T-1h   [ ] Go/no-go call. Criteria: variance <0.01%, 0 unexplained
  22:00  [ ] Disable the legacy scheduled job (SM37)           Basis     5m
  22:05  [ ] Verify no in-flight IDocs (WE02, status 64/51)    Basis     10m
  22:15  [ ] Snapshot target tables (restore point)            DBA       20m
  22:35  [ ] Run full load, batch size 5000                    T.Okafor  2h30m
             Checkpoint every 50k rows. Progress on dashboard X.
             ABORT CRITERIA: error rate >0.5% or elapsed >4h
  01:05  [ ] Run reconciliation report                         T.Okafor  20m
  01:25  [ ] Business spot-check: 20 named invoices            J.Weber   30m
  01:55  [ ] GO/NO-GO #2. If no-go → rollback step R1.
  02:00  [ ] Flip feature flag integration.invoice.source=new  P.Shah    2m
  02:05  [ ] Enable new scheduled job                          P.Shah    5m
  02:10  [ ] Synthetic transaction end to end                  T.Okafor  15m
  02:25  [ ] Monitor 30 min: error rate, lag, DLQ depth        all       30m
  03:00  [ ] FINAL GO/NO-GO. After this point rollback requires
             manual data correction — see R2.
  03:10  [ ] Notify stakeholders, close window                 P.Shah    10m

  ROLLBACK
  R1 (before flag flip, clean):  restore snapshot, re-enable legacy job. 30m.
  R2 (after flag flip):          flip flag back, re-enable legacy job,
                                 reconcile and manually correct anything
                                 written by the new path. 2h + manual work.

  COMMS
  22:00 "Cutover started"   02:10 "Cutover complete, monitoring"
  Every hour: status in #acme-cutover, even if nothing has changed.
```

> ⚠ **The window is shorter than it looks**
>
> A six-hour window is not six hours of execution time. Subtract the go/no-go calls, the business verification, the thirty minutes of monitoring, and the time needed to execute a rollback if you have to. If the rollback takes two hours, your real point of no return is four hours in, and your load must finish well before that. Work backwards from the rollback deadline when you build the plan, and put that deadline in the runbook in bold.

Operations and handover

- [ ] A dashboard per integration: throughput, success rate, latency per hop, DLQ depth, oldest message age, credential expiry, last successful run per feed.
- [ ] Alerts routed to the customer's own queue from day one, each with a runbook linked.
- [ ] Daily reconciliation report emailed to the business owner automatically.
- [ ] A DLQ triage and replay procedure that a support engineer can execute.
- [ ] A credential register: which credential, which system, who owns it, when it expires, how to rotate it.
- [ ] A dependency map: which upstream systems, which maintenance windows, which contacts, which escalation paths.
- [ ] Documented recovery procedures for the five most likely failures, each tested at least once.
- [ ] A handover session where their team diagnoses a fault you inject, using only the artefacts you left, while you stay silent.

Designing an Integration a Customer Will Approve

The best technical design that nobody approves delivers nothing. In an enterprise, approval is a process involving an architect, a security reviewer, a data protection officer, a platform owner and a business sponsor — each of whom has a different question. The integration specification document is the artefact that answers all of them at once, and writing a good one is a genuine engineering skill.

The integration specification

TEXT

INTEGRATION SPECIFICATION – Invoice sync: ACME SAP S/4HANA → DocPipeline
Version 1.3 | Author: P. Shah | Status: For approval | Date: 2026-03-02

1. PURPOSE AND SCOPE
   One paragraph in business language. What business outcome, for whom, why now.
   Explicitly: what is OUT of scope. The out-of-scope list prevents more
   arguments than the in-scope list.

2. SYSTEMS AND OWNERSHIP

System	Role	Environment	Owner	Contact
SAP S/4HANA	Source	PRD cli 100	Finance IT	Basis team
DocPipeline	Target	prod-eu	Vendor (us)	support@ ...
Azure Service Bus	Transport	prod-eu	Platform	#platform

3. DATA CONTRACT
   Entity: Invoice. Volume: 12,000/day avg, 40,000 peak (month end).

Field	Type	Source	Required	Classification	Notes
invoice_id	string	VBELN	Y	Internal	key
vendor_id	string	LIFNR	Y	Internal	
amount_minor	int64	NETWR*100	Y	Confidential	minor units
currency	string	WAERS	Y	Internal	ISO4217
vendor_tax_id	string	STCD1	N	RESTRICTED	NOT transferre

4. FLOW
   Sequence diagram (see appendix A). Trigger, steps, systems, decision points.

5. INTERACTION MODEL
   Direction: SAP → us. Style: asynchronous, event driven via IDoc → queue.
   Frequency: near real time (target < 5 min end to end).
   Initial load: one-time bulk, 2.4M historical invoices, ~6 hours.

6. ERROR HANDLING MATRIX

Condition	Detection	Action	Retry	Alert	Owner
Source unavailable	conn timeout	retry w/ backoff	5	WARN	us
Auth failure	401	stop, alert	0	CRIT	Basis
Schema mismatch	validation	DLQ + alert	0	CRIT	us
Business rejection	BAPI type E	reject queue	0	INFO	Finance
Rate limited	429	backoff, resume	inf	WARN	us
Target unavailable	503	queue, retry	inf	WARN	us
Duplicate	dedupe hit	ignore, count	-	none	-

7. NON-FUNCTIONAL REQUIREMENTS
   Latency: p95 < 5 min source-commit to target-visible.
   Throughput: sustained 50/s, burst 400/s.
   Availability: 99.5% monthly, excluding announced SAP maintenance windows.
   Data retention: queue 14d, DLQ 30d, archive 7 years (regulatory).
   RPO 0 (no data loss, outbox pattern). RTO 4 hours.

```
8. SECURITY
   Identity: technical user SVC_DOCPIPE_PRD, type SYSTEM, roles Z_DOCPIPE_READ.
   Transport: TLS 1.2+, SNC for RFC, mTLS to Service Bus.
   Secrets: Azure Key Vault, workload identity, no secrets in config.
   Network: our subnet 10.42.0.0/20 → sap-gw.acme.corp:44300 (outbound only).
   Data classification: Confidential. No PII crosses the boundary (see sec 3).
   Audit: every read logged with user, timestamp, record count.

9. OPERATIONS
   Monitoring: dashboard URL, alert routing to ACME ServiceNow queue INT-OPS.
   Runbooks: 6 documented procedures, linked from each alert.
   Reconciliation: daily 06:00, report to finance-ops@acme.
   Support: L1 ACME service desk, L2 ACME integration team, L3 vendor (4h SLA).

10. DEPENDENCIES, RISKS AND ASSUMPTIONS
   | Risk                                | Impact | Likelihood | Mitigation |
   | SAP maintenance window unannounced  | High   | Medium     | queue absorbs; alert
   | Volume exceeds estimate at year end | Medium | High       | autoscale consumers,
   | Technical user password expiry      | High   | Medium     | type SYSTEM (no expir

11. TESTING AND CUTOVER
   Test plan summary, environments, dress rehearsal dates, go/no-go criteria.

12. OPEN QUESTIONS
   Keep this section. An honest open-questions list builds far more trust than
   a document that pretends everything is decided.

APPENDIX A: sequence diagram
APPENDIX B: field mapping (full)
APPENDIX C: sample payloads
```

✦ **The error-handling matrix is the section that gets you approved**

Architects and operations managers have been burned by integrations that work in the happy path and fail mysteriously otherwise. A table that enumerates every failure condition, who detects it, what happens, and crucially **who owns it**, demonstrates that you have thought about the three years after go-live rather than just the launch. In my experience it is the single section most likely to be read closely, and the one that most changes how seriously you are taken.

The kickoff questions

ABOUT THE DATA

What is the system of record? What volume, what peak, what growth? How far back does history go and do we need it? What is the natural key? How are deletes represented? What are the known data quality problems? Which fields are regulated?

ABOUT THE INTERFACE

Which APIs exist and which are activated? Who owns them? Is there an existing integration we can model on? What are the rate limits and who else consumes them? What is the sandbox and how closely does it match production?

ABOUT TIMING

How fresh does the data need to be, really? When are the maintenance windows? When is month-end/quarter-end and what happens to volume? When is the change freeze? How many change windows do we get?

ABOUT FAILURE

What happens today when this breaks? Who notices? What is an acceptable amount of lateness? Is duplicate processing dangerous, and how dangerous? Is partial data worse than no data?

ABOUT PEOPLE

Who approves the design? Who approves security? Who operates it after go-live? Who is the business owner who will receive the reconciliation report? Who do we call at 2am, and do they know that?

ABOUT CONSTRAINTS

What must not leave the country, the datacentre, the network? What is the egress policy? What authentication mechanisms are permitted? Is there a mandated integration platform? What is the lead time for a firewall rule?

✓ **Draw the sequence diagram in the room**

In the kickoff, draw the flow on a whiteboard with the customer's people, box by box, and ask at each arrow: what happens if this fails? Three things happen. You surface the systems nobody mentioned. You discover that two people in the room disagree about how the current process works — which is extremely common and extremely valuable to find early. And you establish yourself as someone who thinks about failure, which is the reputation that gets your designs approved quickly for the rest of the engagement.

Interview Questions

Why is so much of a deployment taken up by integration work, and how do you plan for it?

Because the valuable part of any enterprise system is the data that already exists, and that data lives in systems that were built over decades by different teams with different assumptions. Getting it out means navigating an API that may not be activated, a service account that takes three weeks to provision, a firewall rule owned by a team with its own queue, a schema that nobody fully documents, and a change process measured in quarters. None of that is visible in a demo, which is why demos take four weeks and deployments take nine months. I plan for it by counting integration points rather than features during scoping, because each point brings a credential that expires, a schema that drifts, a rate limit, a maintenance window and a team to coordinate with — and the effort is superlinear in the number of points, not linear. Then in week one I establish, in writing, the answers to six questions per integration: what is the system of record, how do I get data out and who owns that interface, what is the volume and shape, how fresh does it need to be, what happens when it fails, and what is the lead time for a change. Those answers shape the architecture far more than any technology preference, and getting them early is the single biggest determinant of whether the project lands on time.

Walk me through the four integration styles and when you would choose each.

File transfer: one system writes a file, another reads it. It has no shared runtime, works through any firewall, is trivially auditable and replayable, and it remains the most common inter-company integration in the world. I choose it when the counterparty is another organisation, when latency of minutes to hours is acceptable, or when it is simply what the customer already does well. Shared database: two applications use the same tables. Consistent and fast, but it is total coupling — the

schema becomes a public API nobody can version, and within two years nobody dares change a column. I avoid it except for read-only access to a replica or a view that the owning team maintains deliberately as an interface. RPC — REST, SOAP, gRPC, SAP RFC: one system calls another and waits. Simple mental model, immediate result, good for queries. The hidden cost is temporal coupling: both systems must be up simultaneously, so a four-hour maintenance window downstream becomes four hours of failures. And it makes a network call look like a local one, which hides partial failure and timeout ambiguity. Messaging: a broker decouples producer and consumer in time, space and throughput. I default to this for anything that can be asynchronous, because one component's outage degrades the system rather than destroying it, and the queue absorbs bursts. The costs are real — you need a broker, duplicates are a fact of life, and debugging requires distributed tracing — but they are costs I know how to pay. Most real flows combine two or three styles, and the skill is choosing the boundary between them deliberately.

What is idempotency and how do you implement it?

An operation is idempotent if performing it more than once has the same effect as performing it once. It matters because essentially every realistic delivery mechanism is at-least-once: brokers redeliver on timeout, clients retry on an ambiguous response, operators replay a dead letter queue, and a file job gets rerun. Without idempotency, each of those produces a duplicate business transaction. There are four implementations I use in order of preference. First, natural idempotence: structure the write as an upsert on a business key, so re-running it converges to the same state. Second, a dedupe store: record the message ID in Redis or a table with SETNX and a TTL, and skip anything already seen — necessary for operations that are not naturally idempotent, like sending an email or incrementing a balance. Third, idempotency keys on outbound calls, where the provider dedupes server-side — and critically, the key must be derived from business identifiers so that it is stable across retries of the same logical operation, not generated fresh at call time, which is a mistake I have seen more than once. Fourth, conditional writes with optimistic concurrency, which is the safest for state machines because it also protects against out-of-order updates. Two details people miss: the dedupe TTL must exceed the maximum possible redelivery window, including DLQ retention and manual replay, so seven days is a sensible floor rather than one hour; and idempotency has a scope that must be documented — per tenant, for how long, keyed on what, and what happens if the same key arrives with a different payload.

Explain the transactional outbox pattern and the problem it solves.

It solves the dual-write problem. When a business operation needs to update your database and publish an event, there is no ordering that is safe, because the two systems have no shared transaction. If you commit the database then publish, a crash in between means the state changed but nobody was told. If you publish then commit, a crash means the world believes something happened that did not. The outbox pattern makes the publication part of the database transaction: in the same transaction that updates the invoice, you insert a row into an outbox table containing the event. The transaction either commits both or neither, so the event can never be lost. A separate relay process then reads unpublished outbox rows and publishes them to the broker, marking them published afterwards. The relay can be a simple poller using SELECT ... FOR UPDATE SKIP LOCKED so

multiple instances can run safely, or it can be Debezium reading the database's write-ahead log, which removes the polling latency. The important property to be honest about is that this gives you at-least-once, not exactly-once: if the relay crashes after the broker acknowledges but before it marks the row published, the event is published twice. That is correct and expected — the outbox guarantees no lost events, and consumer idempotency handles the duplicates. The other thing people forget is housekeeping: the outbox table grows forever unless you delete published rows on a schedule, in batches, and I have seen one reach four hundred gigabytes.

A customer says their API has a rate limit you will exceed. What do you do?

First I measure precisely what the budget is and what is already consuming it, because in an enterprise the limit is usually shared across every integration in the company and I need to know my share, not the total. For Salesforce that means calling the limits endpoint and reading the Sforce-Limit-Info header; for Graph it means understanding per-workload throttling; for an internal system it often means asking the platform owner, because it is undocumented. Then I attack the volume rather than the limit. The biggest lever is almost always using the bulk path instead of per-record calls: one Bulk API 2.0 job for a million records versus a million REST calls is a six-order-of-magnitude difference and usually decides whether the project is feasible. Next is going incremental — delta queries, change streams, modified-since filters — so a full scan happens once at go-live and never again. Then caching reference data that changes daily at most, batching with composite or $batch endpoints, and debouncing so a record that changes five times in a minute syncs once. On the client side I implement a token-bucket limiter matched to their published rate, plus adaptive concurrency using additive-increase multiplicative-decrease to discover the real limit where it is undocumented, and I always honour Retry-After because enterprise APIs mean it and ignoring it escalates throttling. Finally I instrument consumption as a percentage of budget, alert at seventy per cent, and proactively show the platform owner my dashboard. That last part is political rather than technical, but quota is a shared commons and the team that treats it carelessly is the team that gets throttled, audited and eventually removed.

How would you approach integrating with a customer's SAP system?

I would start with vocabulary and people rather than code, because in SAP the gatekeepers are the Basis team and they can tell within ninety seconds whether you know what you are asking for. I would establish which system and client we are talking to, whether it is ECC or S/4HANA, and whether SAP Gateway is installed and which OData services are activated — because an OData path is dramatically easier than RFC. Then I would ask for a technical user of type SYSTEM or COMMUNICATION, never DIALOG, because a dialog user consumes a named licence, is subject to password expiry and interactive login policies, and will lock your integration out at 2am on day ninety. I would ask what authorisation roles it gets and request an authorisation trace during testing, because "it returns no rows" in SAP is far more often a permissions problem than a code problem. Technically there are five ways in: OData via Gateway, which is my first choice; RFC and BAPI calls, which need the NetWeaver RFC SDK, gateway ports and often SNC encryption; IDocs, which are asynchronous, status-tracked and the most reliable mechanism, especially for high-volume flows; CDS views and direct HANA reads; and SAP Integration Suite if they have it, in which case they will probably require me to route through it.

Three mistakes I specifically avoid: forgetting BAPI_TRANSACTION_COMMIT, which makes everything report success while saving nothing; reading tables directly instead of using an API, which bypasses business logic and breaks on the next support pack; and assuming HTTP 200 means success, when BAPIs return errors inside the RETURN table of a successful response.

How do you design a webhook consumer that does not lose data?

Six things, in order. Verify the signature first, using HMAC over the raw request body with a constant-time comparison, before parsing anything — the endpoint is reachable by anyone who can reach your gateway, and re-serialising parsed JSON before verifying will break the signature because key order changes. Check the timestamp and reject anything outside a few minutes, because a valid signature on a replayed payload from last week is still valid. Return 200 fast and do the work asynchronously: most providers time out at three to ten seconds and will disable a slow endpoint, so I validate, write to a queue or outbox, and return in under a hundred milliseconds. Deduplicate on the provider's event ID with a TTL longer than their maximum retry window, because every provider delivers duplicates eventually. Treat the payload as a hint rather than truth — webhooks arrive out of order, so I compare a version or updated-at field and discard anything older than what I already have, and for critical data I fetch current state from the API rather than trusting the body. And finally, build a reconciliation path: a periodic poll of everything changed since a timestamp, because webhooks will be missed when the provider has an outage, when my endpoint is down past their retry limit, or when a firewall rule changes. That poll is not optional; it is the safety net that turns "we lost four hours of events and nobody noticed" into "we caught up automatically".

What is the difference between a queue and a log, and how do you choose?

In a queue — RabbitMQ, SQS, Service Bus, IBM MQ — a message is consumed and removed. The broker tracks per-message state: unacknowledged, acknowledged, dead-lettered. You get rich routing, priorities, TTLs, scheduled delivery and built-in dead-letter semantics, and competing consumers give you horizontal scaling. What you do not get is replay: once consumed, the message is gone. In a log — Kafka, Pulsar, Kinesis, Event Hubs — messages are appended to an immutable, ordered, partitioned sequence, and consumers track their own offsets. Reading does not remove anything, so any number of independent consumer groups can read the same data, and replay is simply resetting an offset. What you do not get out of the box is per-message retry and dead-lettering; you build those. I choose with three questions. Will more than one independent system need this data, now or in the future? If yes, a log, because adding a consumer group requires nothing from the producing team, which in an enterprise is a significant political advantage. Will I need to reprocess history after fixing a bug or to rebuild a downstream store? If yes, a log. Does each item need to be worked on exactly once by one worker, with per-message retry, delay and dead-lettering? If yes, a queue. In practice the customer already owns one or both and the real decision is using whichever they have correctly — the dangerous failure is designing for log semantics on top of a queue, or assuming queue-style per-message retry in Kafka, where a poison message blocks an entire partition unless you explicitly route it to a DLQ topic.

What do you think about exactly-once delivery?

It is mostly a marketing term, and being clear about that is a professional obligation. Kafka's exactly-once semantics are real but narrow: they cover a read-process-write cycle entirely within Kafka, using transactions and read-committed isolation. The moment a consumer writes to an external system — a database, SAP, an HTTP API — you are back to at-least-once, because there is no distributed transaction spanning Kafka and SAP, and two-phase commit across heterogeneous enterprise systems is not something anyone will let you deploy. So the correct engineering position is: assume at-least-once delivery and make every consumer idempotent. That combination is sometimes called effectively-once, and it is achievable, testable and explainable. The reason to be firm about this with customers is that promising exactly-once is a promise you cannot keep, and the promise gets examined on the day a network blip causes a duplicate posting in their general ledger. I would rather have the harder conversation in the design review, where I can show the dedupe mechanism and the reconciliation report, than in an incident review three months later.

A customer wants to give you direct read access to their production database. How do you respond?

Carefully, because it sounds helpful and is usually a trap. An application's tables are its internal state, not its API: the status column means something only in combination with three other tables and logic implemented in code, so reading directly means reimplementing that logic and getting it subtly wrong. The schema is not a contract — the owning team will rename a column in a routine release because from their perspective it is private, and they are right — so my integration breaks and the postmortem concludes it was my fault. And polling on updated_at misses hard deletes entirely and depends on a timestamp that is often unreliable, especially after bulk updates that bypass triggers. So I would ask three questions: is there an API, even an imperfect one; can the owning team expose a view or materialised view as a deliberate, versioned interface; and can we use log-based change data capture against a replica instead? CDC with Debezium reads the transaction log, captures inserts, updates and deletes in commit order with before and after images, and costs the source database almost nothing. But I would raise the replication-slot risk explicitly and in writing with the DBA, because in PostgreSQL a stalled consumer causes WAL to accumulate until the source database's disk fills and production stops accepting writes — which is a real way to take down a system you were only supposed to read from. The mitigations are monitoring slot lag, setting max_slot_wal_keep_size, configuring a heartbeat query, and having a documented re-snapshot runbook. If direct access really is the only option, I would insist on a read-only replica, a dedicated login, a statement timeout, agreed query patterns, and written acknowledgement that the schema may change without notice.

How do you plan a cutover for an integration that replaces an existing process?

The organising principle is that I must be able to roll back, and if I cannot roll back I do not have a plan, I have a hope. Concretely: I profile the source data early, because I will find that several per cent of records violate a constraint everyone assured me was enforced, and I want that in week two rather than in the window. I make the migration idempotent and resumable with checkpointing, because it will fail halfway at least once. I rehearse at full scale twice — a load that takes ninety minutes with ten per cent of the data takes fifteen hours with all of it, and the second rehearsal is where I find the problems the first rehearsal's fixes introduced. I run the new integration in parallel, writing to a

> shadow target while the old process stays authoritative, and I produce a daily comparison report where every variance must be explained rather than waved away. I agree go/no-go criteria in writing in advance, when everyone is calm, not at two in the morning. I switch with a feature flag rather than a deployment, because a flag flips back in seconds. I get a change freeze around the window agreed with their change board. And I write the runbook with an owner and a duration on every line, work backwards from the rollback deadline to find the real point of no return, and put that in bold. Afterwards, two weeks of hypercare with daily reconciliation, the rollback still available, and that is when I train their support team, because the questions they ask are real ones.

> **What goes into an integration specification, and which part matters most?**
> Purpose and scope in business language, with an explicit out-of-scope list that prevents more arguments than the in-scope list. Systems and ownership, naming the actual owner and contact for each. The data contract: every field, its type, its source, whether it is required, and its data classification — which is the section the data protection officer reads. The flow, with a sequence diagram. The interaction model: direction, synchronous or asynchronous, frequency, and the one-time initial load separately from steady state. Non-functional requirements with actual numbers: latency target, throughput sustained and burst, availability with named exclusions, retention, RPO and RTO. Security: the identity mechanism, transport encryption, secret storage, network path, audit. Operations: dashboards, alert routing, runbooks, reconciliation, support tiers and escalation. Risks and assumptions with mitigations. And an open-questions section, which I keep deliberately, because an honest list of what is not yet decided builds far more trust than a document pretending everything is settled. The part that matters most is the error-handling matrix: a table enumerating every failure condition, how it is detected, what action is taken, whether it retries, what severity of alert it raises, and who owns it. Architects and operations managers have all been burned by integrations that work beautifully in the happy path and fail mysteriously otherwise, and that table is what demonstrates you have thought about the three years after go-live rather than just the launch. In my experience it is the section read most closely and the one that most changes how seriously the design is taken.

Integration Design Checklist and Lab Plan

A checklist to run before any integration goes live, and a seven-part lab that builds a realistic enterprise integration end to end on a laptop. The checklist is the deliverable; the lab is how it becomes instinct.

Design checklist

CONTRACT AND DATA

- ☐ System of record identified for every entity; arbitration rule agreed where two systems claim it.

- [ ] Field-level mapping documented, including type, source, required flag and data classification.
- [ ] Natural/business key identified and confirmed stable. It is the basis of your idempotency.
- [ ] Volume documented: average, peak, growth, record size, history depth.
- [ ] Deletes: how are they represented, and does the target need them?
- [ ] Money as integer minor units with an ISO-4217 currency field. No floats. No assumed two decimal places.
- [ ] Timestamps are timezone-aware UTC with the business event time distinct from the message time.
- [ ] Schema versioned, with an explicit compatibility policy and a parallel-run plan for the next version.
- [ ] Known data quality problems profiled and documented before design, not discovered at cutover.

RELIABILITY

- [ ] Delivery semantics stated explicitly; consumers idempotent; dedupe scope and TTL documented.
- [ ] Retry policy: what is retryable, how many attempts, exponential backoff with full jitter, a global retry budget.
- [ ] Retry-After honoured everywhere.
- [ ] Timeouts explicit at every layer, forming a budget where inner timeouts are shorter than outer ones.
- [ ] Circuit breaker on every external dependency, tripping on latency as well as errors.
- [ ] Dead letter channel with an owner, an alert, a triage process and a tested replay tool.
- [ ] Transactional outbox or equivalent wherever a state change and a published event must agree.
- [ ] Reconciliation job comparing source and target counts and amounts, with an unexplained-variance alert.
- [ ] Deadline monitoring: alert when an expected file or batch does not arrive.
- [ ] Claim check for payloads above the broker limit, with blob lifetime exceeding queue plus DLQ retention.

SECURITY AND GOVERNANCE

- [ ] Dedicated service identity, least privilege, owned by a team, documented in the customer's register.
- [ ] No integration runs as a named human, ever.
- [ ] Secrets in the customer's vault; workload identity preferred; nothing in images, config or git.
- [ ] Credential expiry exposed as a metric with alerts at 30, 14 and 7 days, sized to the renewal lead time.
- [ ] TLS verification never disabled; internal CAs trusted explicitly; proxy variables set for every runtime.
- [ ] Data classification table completed; minimisation applied; restricted fields explicitly excluded.
- [ ] Residency, lawful basis and sub-processor questions raised in week one, not at go-live.
- [ ] Audit trail for reads and writes; erasure path understood across queues, DLQs, archives and logs.

OPERATIONS AND HANDOVER

- [ ] Dashboard per integration: throughput, success rate, per-hop latency, DLQ depth, oldest message, lag, credential expiry.
- [ ] Alerts routed to the customer's queue from day one, each with a runbook link and a business-impact description.
- [ ] Correlation ID propagated end to end and visible in every system's logs.
- [ ] Daily reconciliation report delivered automatically to the business owner.
- [ ] Credential register and dependency map handed over, including upstream maintenance windows and contacts.
- [ ] Rate-limit consumption instrumented as a percentage of agreed budget, alerting at 70%.
- [ ] Handover acceptance test completed: their team diagnoses an injected fault using only your artefacts.

Lab plan — build a realistic enterprise integration

1. **Lab 1 — The file drop that survives reality (3 hours)**
   Build a consumer for a CSV drop with every guard: sentinel file, stability check, atomic claim by rename, batch-level idempotency keyed on a content hash, control-file count and checksum

validation, header schema assertion, a reject threshold, and a single transaction per file. Then attack it: feed it a file still being written, the same file twice, a file with a column inserted in the middle, a UTF-8 BOM, CRLF endings, a field containing an unescaped comma, a zero-byte file, and no file at all. Success criterion: every one of those produces a clear, actionable log line and the correct outcome, and the missing-file case raises an alert.

2

Lab 2 — Queue versus log, hands on (4 hours)

Run RabbitMQ and Kafka in Docker. Publish the same event stream to both. In RabbitMQ, implement a dead letter exchange, demonstrate redelivery on nack, and implement a manual retry with a delay queue. In Kafka, implement manual offset commits after processing, route a poison message to a DLQ topic, and demonstrate that failing to do so blocks the partition. Then add a second consumer group to Kafka and show it reads all history independently, and try the equivalent in RabbitMQ to feel the difference. Finally, deliberately key your Kafka producer by tenant ID with one dominant tenant and observe the hot partition in the per-partition lag graph. Success criterion: you can explain from your own graphs why the partition key choice matters.

3

Lab 3 — Idempotency and the outbox (3 hours)

Build a service with a POST endpoint that writes a business record and publishes an event, using the transactional outbox pattern with a polling relay using SELECT FOR UPDATE SKIP LOCKED. Then break it on purpose: kill the relay between the broker flush and the published-at update, and confirm the event is republished and that your idempotent consumer handles it. Kill the process between the business write and the publish, and confirm nothing is lost. Send the same request twice with the same Idempotency-Key and confirm one job is created. Send the same key with a different payload and confirm a 409. Success criterion: you can state precisely which guarantee each mechanism provides and demonstrate each one.

4

Lab 4 — Webhooks in both directions (3 hours)

Build a consumer with HMAC verification over the raw body, a timestamp replay window, constant-time comparison, dedupe on event ID, and a sub-100ms response that enqueues the work. Write tests that send a tampered body, a stale timestamp, a replayed event and an out-of-order update, and assert the correct rejection or suppression for each. Then build the producer side: signed delivery, an exponential retry schedule with jitter, permanent-versus-transient status classification, a delivery log, auto-disable after sustained failure with customer notification, and a replay endpoint. Success criterion: you can take your own consumer down for twenty minutes and lose nothing once it returns.

5

Lab 5 — CDC and the hot path (4 hours)

Stand up PostgreSQL, Kafka Connect and Debezium. Capture changes from an `invoices` table. Observe the initial snapshot, then inserts, updates and deletes. Translate the raw row-change events into named business events in a single translation step and publish to a clean topic. Then cause the failure that matters: stop the connector, generate load, and watch `pg_replication_slots` lag and WAL size grow. Set `max_slot_wal_keep_size` and confirm

the database protects itself. Write the runbook for dropping and recreating the slot. Success criterion: you can explain the replication-slot risk to a DBA in two minutes and show them your monitoring for it.

6

Lab 6 — Living within a rate limit (3 hours)

Build a mock upstream with WireMock that enforces a token-bucket limit of eight requests per second with a burst of twenty, returns 429 with Retry-After when exceeded, and randomly injects 503s and five-second delays. Write a client that stays within the limit using a token bucket, discovers an undocumented concurrency ceiling with additive-increase multiplicative-decrease, honours Retry-After, retries with full jitter, trips a circuit breaker on sustained failure, and exposes quota consumption as a metric. Load it with 100,000 requests. Success criterion: zero sustained 429s, a graph showing your concurrency converging on the real limit, and a completion time within twenty per cent of the theoretical minimum.

7

Lab 7 — The full pipeline with a cutover (5 hours)

Compose everything: an SFTP drop feeding a file consumer, which publishes via an outbox to Kafka, consumed by a translator that enriches from a cached reference API, routed by region, written idempotently to a target database, with a DLQ, a daily reconciliation job, correlation IDs propagated end to end and visible in every log, and a dashboard showing throughput, lag, DLQ depth and reconciliation variance. Then perform a cutover: run a legacy path and the new path in parallel against the same input, produce a daily comparison report, define go/no-go criteria, switch with a feature flag, and practise the rollback. Finally, write the integration specification for what you built, including the error-handling matrix, and hand the whole thing to a colleague. Success criterion: they can, from your specification and dashboards alone, diagnose a fault you inject and execute the documented recovery without asking you a question.

✓ **The habit that makes you the person they ask for**

After every integration incident, add a fixture file. The malformed payload, the unexpected 200-with-an-error, the HTML login page where JSON was expected, the truncated response, the duplicate that arrived nine days later. Over a couple of years that directory becomes the most valuable artefact you own: a concrete record of every way enterprise systems actually fail, encoded as executable tests. It is also the thing that lets you say, in a design review, "I have seen this fail three ways and here is how we handle each" — which is the sentence that gets your design approved and gets you asked for by name on the next engagement.